

XGBoost

Zach Skiba

2026-04-26

Section 0 — Setup & Package Loading

Overview

We load all required packages and set a global seed for reproducibility. Key packages: `xgboost` for gradient boosted trees, `SHAPforxgboost` for SHAP value computation and visualization, `caret` for confusion matrix utilities, `pROC` for ROC-AUC curves, and `ggplot2` / `kableExtra` for visualization and reporting.

The seed is set once here and repeated in any chunk that involves randomness to ensure reproducibility across non-sequential chunk execution.

```
required_packages <- c(
  "xgboost", "SHAPforxgboost", "caret", "pROC",
  "ggplot2", "kableExtra", "dplyr", "tidyr", "Matrix"
)

installed <- rownames(installed.packages())
to_install <- setdiff(required_packages, installed)
if (length(to_install) > 0) install.packages(to_install, dependencies = TRUE)

library(xgboost)
```

```
## Warning: package 'xgboost' was built under R version 4.5.3
```

```
library(SHAPforxgboost)
```

```
## Warning: package 'SHAPforxgboost' was built under R version 4.5.3
```

```
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
```

```
##
## Attaching package: 'pROC'
```

```
## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

```
library(ggplot2)
library(kableExtra)
```

```
## Warning: package 'kableExtra' was built under R version 4.5.3
```

```
library(dplyr)
```

```
##  
## Attaching package: 'dplyr'
```

```
## The following object is masked from 'package:kableExtra':  
##  
##     group_rows
```

```
## The following objects are masked from 'package:stats':  
##  
##     filter, lag
```

```
## The following objects are masked from 'package:base':  
##  
##     intersect, setdiff, setequal, union
```

```
library(tidyr)  
library(Matrix)
```

```
##  
## Attaching package: 'Matrix'
```

```
## The following objects are masked from 'package:tidyr':  
##  
##     expand, pack, unpack
```

```
set.seed(571)  
  
cat("All packages loaded successfully.\n")
```

```
## All packages loaded successfully.
```

```
cat("xgboost version:", as.character(packageVersion("xgboost")), "\n")
```

```
## xgboost version: 3.2.1.1
```

Section 1 — Data Loading & Validation

Overview

We load all four pre-split CSVs directly — no re-splitting is done here. The data was stratified 80/20 by a teammate. We verify dimensions, column alignment, missing values, and class distributions for both targets.

Unlike the logistic regression notebook, we do NOT convert targets to factors here — XGBoost requires numeric labels (0/1 for binary, 0/1/2 for multiclass).

```
binary_train    <- read.csv("../data/processed/binary_train.csv")  
binary_test     <- read.csv("../data/processed/binary_test.csv")  
multiclass_train <- read.csv("../data/processed/multiclass_train.csv")  
multiclass_test  <- read.csv("../data/processed/multiclass_test.csv")
```

```
# Dimensions  
cat("binary_train:  ", nrow(binary_train),      "x", ncol(binary_train),      "\n")
```

```
## binary_train:      202943 x 22
```

```
cat("binary_test:      ", nrow(binary_test),      "x", ncol(binary_test),      "\n")
```

```
## binary_test:      50737 x 22
```

```
cat("multiclass_train:", nrow(multiclass_train), "x", ncol(multiclass_train), "\n")
```

```
## multiclass_train: 202943 x 22
```

```
cat("multiclass_test: ", nrow(multiclass_test), "x", ncol(multiclass_test), "\n")
```

```
## multiclass_test:  50737 x 22
```

```
# Column alignment  
cat("\nColumns match (binary):      ",  
    identical(colnames(binary_train), colnames(binary_test)), "\n")
```

```
##  
## Columns match (binary):      TRUE
```

```
cat("Columns match (multiclass):",  
    identical(colnames(multiclass_train), colnames(multiclass_test)), "\n")
```

```
## Columns match (multiclass): TRUE
```

```
# Missing values  
cat("\nNA counts:\n")
```

```
##  
## NA counts:
```

```
cat("binary_train:",      sum(is.na(binary_train)),      "\n")
```

```
## binary_train: 0
```

```
cat("binary_test:",      sum(is.na(binary_test)),      "\n")
```

```
## binary_test: 0
```

```
cat("multiclass_train:", sum(is.na(multiclass_train)), "\n")
```

```
## multiclass_train: 0
```

```
cat("multiclass_test:", sum(is.na(multiclass_test)), "\n")
```

```
## multiclass_test: 0
```

```
cat("Binary target distribution (train):\n")
```

```
## Binary target distribution (train):
```

```
print(round(prop.table(table(binary_train$Diabetes_binary)) * 100, 2))
```

```
##  
##      0      1  
## 86.07 13.93
```

```
cat("\nMulticlass target distribution (train):\n")
```

```
##  
## Multiclass target distribution (train):
```

```
print(round(prop.table(table(multiclass_train$Diabetes_012)) * 100, 2))
```

```
##  
##      0      1      2  
## 84.25  1.82 13.93
```

The four datasets load cleanly with no missing values. Dimensions and column alignment are confirmed across train and test sets. Class imbalance is present in both tasks — 86/14 for binary and 84/2/14 for multiclass — consistent with the EDA findings.

```
set.seed(571)
```

```
scale_cols <- c("BMI", "MentHlth", "PhysHlth")
```

```
train_means <- colMeans(binary_train[, scale_cols])  
train_sds    <- apply(binary_train[, scale_cols], 2, sd)
```

```
binary_train[, scale_cols] <- scale(binary_train[, scale_cols],  
                                     center = train_means, scale = train_sds)  
binary_test[, scale_cols]  <- scale(binary_test[, scale_cols],  
                                     center = train_means, scale = train_sds)
```

```
mc_means <- colMeans(multiclass_train[, scale_cols])  
mc_sds   <- apply(multiclass_train[, scale_cols], 2, sd)
```

```
multiclass_train[, scale_cols] <- scale(multiclass_train[, scale_cols],  
                                         center = mc_means, scale = mc_sds)  
multiclass_test[, scale_cols]  <- scale(multiclass_test[, scale_cols],  
                                         center = mc_means, scale = mc_sds)
```

```
spw <- sum(binary_train$Diabetes_binary == 0) / sum(binary_train$Diabetes_binary == 1)  
cat("scale_pos_weight:", round(spw, 4), "\n")
```

```
## scale_pos_weight: 6.1772
```



```

train_idx <- createDataPartition(binary_train$Diabetes_binary, p = 0.9, list = FALSE)
tune_data  <- binary_train[ train_idx, ]
val_data   <- binary_train[-train_idx, ]

X_tune <- as.matrix(tune_data[, setdiff(names(tune_data), "Diabetes_binary")])
y_tune <- tune_data$Diabetes_binary
X_val  <- as.matrix(val_data[, setdiff(names(val_data), "Diabetes_binary")])
y_val  <- val_data$Diabetes_binary

dtune <- xgb.DMatrix(data = X_tune, label = y_tune)
dval  <- xgb.DMatrix(data = X_val, label = y_val)

X_train <- as.matrix(binary_train[, setdiff(names(binary_train), "Diabetes_binary")])
y_train <- binary_train$Diabetes_binary

cat("Training observations:", nrow(binary_train), "\n")

```

```
## Training observations: 202943
```

```
cat("Tuning slice:      ", nrow(tune_data), "\n")
```

```
## Tuning slice:      182649
```

```
cat("Validation slice:   ", nrow(val_data), "\n")
```

```
## Validation slice:   20294
```

Section 2 — Binary XGBoost

Overview

We train an XGBoost classifier on the binary task (`Diabetes_binary` : 0 = no diabetes, 1 = prediabetes/diabetes). XGBoost is a gradient boosted tree ensemble that builds trees sequentially, each one correcting the residual errors of the previous. Unlike logistic regression, it naturally captures nonlinear relationships and feature interactions without any explicit specification.

Imbalance handling: We use `scale_pos_weight`, XGBoost's native parameter for binary class imbalance. It is set to the ratio of negative to positive class counts in the training data, which tells the loss function to penalize minority class misclassifications proportionally more. This is equivalent to class weighting in `sklearn` and is the recommended approach for tree-based models — no synthetic data generation is needed or appropriate here.

Tuning strategy: We use a stratified 90/10 validation slice carved from the training set. XGBoost's `early_stopping_rounds` monitors val AUC after every boosting round and halts when it stops improving, automatically selecting the optimal number of trees. We then perform a manual grid search over `max_depth` and `eta` (learning rate) — the two parameters with the largest practical impact on XGBoost performance. `subsample` and `colsample_bytree` are set to 0.8 throughout, which is a well-established default that provides regularization through stochastic sampling. After selecting the best hyperparameters, the final model is retrained on the full training set before test evaluation.

Why we do not use cross-validation: XGBoost's early stopping on a validation watchlist serves the same role as CV for simpler models — it provides an honest generalization estimate and prevents overfitting by stopping tree growth at the right point. Running full k-fold CV on XGBoost at this data size would be computationally expensive without meaningfully improving hyperparameter selection.

```
set.seed(571)

# 1. Expanded hyperparameter grid
param_grid <- expand.grid(
  max_depth      = c(4, 6),
  eta            = c(0.03, 0.1),
  min_child_weight = c(1, 10),
  subsample      = c(0.6, 0.8),
  colsample_bytree = c(0.6, 0.8),
  gamma          = c(0, 0.1)
)

cat("Total combinations:", nrow(param_grid), "\n\n")
```

```
## Total combinations: 64
```

```
# 2. repeated evaluation seeds
```

```
seeds <- c(571, 123, 999)
```

```
results <- data.frame()
```

```
for (i in seq_len(nrow(param_grid))) {
```

```
  auc_vals <- c()
```

```
  for (s in seeds) {
```

```
    set.seed(s)
```

```
    params <- list(
```

```
      objective      = "binary:logistic",
```

```
      eval_metric     = "auc",
```

```
      scale_pos_weight = spw,
```

```
      max_depth       = param_grid$max_depth[i],
```

```
      eta              = param_grid$eta[i],
```

```
      min_child_weight = param_grid$min_child_weight[i],
```

```
      subsample        = param_grid$subsample[i],
```

```
      colsample_bytree = param_grid$colsample_bytree[i],
```

```
      gamma            = param_grid$gamma[i],
```

```
      seed             = s
```

```
    )
```

```
    model <- xgb.train(
```

```
      params = params,
```

```
      data   = dtune,
```

```
      nrounds = 1000,
```

```
      evals   = list(val = dval),
```

```
      early_stopping_rounds = 30,
```

```
      verbose = 0
```

```
    )
```

```
    eval_log <- attributes(model)$evaluation_log
```

```
    if (!is.null(eval_log) && "val_auc" %in% colnames(eval_log)) {
```

```
      auc_vals <- c(auc_vals, max(eval_log$val_auc))
```

```
    }
```

```
  }
```

```
# 3. aggregate performance
```

```
mean_auc <- mean(auc_vals, na.rm = TRUE)
```

```
auc_sd   <- sd(auc_vals, na.rm = TRUE)
```

```
n <- length(auc_vals)
```

```
ci_half_width <- 1.96 * auc_sd / sqrt(n)
```

```
ci_lower <- mean_auc - ci_half_width
```

```
ci_upper <- mean_auc + ci_half_width
```

```
# 4. store results
```

```
results <- rbind(results, data.frame(
```

```
  max_depth      = param_grid$max_depth[i],
```

```
  eta            = param_grid$eta[i],
```

```
  min_child_weight = param_grid$min_child_weight[i],
```

```
  subsample       = param_grid$subsample[i],
```

```
  colsample_bytree = param_grid$colsample_bytree[i],
```

```
  gamma          = param_grid$gamma[i],
```

```
  mean_auc       = round(mean_auc, 5),
```

```
  std_auc        = round(auc_sd, 5),
```

```
  ci_lower       = round(ci_lower, 5),
```

```
  ci_upper       = round(ci_upper, 5)
```

```
))
```

```
# 5. logging
cat(sprintf(
  "Combo %02d/%02d | depth=%d eta=%.2f mcw=%d subs=%.1f col=%.1f gamma=%.1f | AUC=%.5f ± %.5f | CI [%.5f, %.5f]\n",
  i, nrow(param_grid),
  param_grid$max_depth[i],
  param_grid$eta[i],
  param_grid$min_child_weight[i],
  param_grid$subsampling[i],
  param_grid$colsample_bytree[i],
  param_grid$gamma[i],
  mean_auc,
  auc_sd,
  ci_lower,
  ci_upper
))
}
```

## Combo 01/64	depth=4	eta=0.03	mcw=1	subs=0.6	col=0.6	gamma=0.0	AUC=0.82763 ± 0.00009	CI [0.82753, 0.82773]
## Combo 02/64	depth=6	eta=0.03	mcw=1	subs=0.6	col=0.6	gamma=0.0	AUC=0.82753 ± 0.00013	CI [0.82739, 0.82767]
## Combo 03/64	depth=4	eta=0.10	mcw=1	subs=0.6	col=0.6	gamma=0.0	AUC=0.82763 ± 0.00019	CI [0.82741, 0.82785]
## Combo 04/64	depth=6	eta=0.10	mcw=1	subs=0.6	col=0.6	gamma=0.0	AUC=0.82706 ± 0.00019	CI [0.82685, 0.82727]
## Combo 05/64	depth=4	eta=0.03	mcw=10	subs=0.6	col=0.6	gamma=0.0	AUC=0.82764 ± 0.00017	CI [0.82745, 0.82783]
## Combo 06/64	depth=6	eta=0.03	mcw=10	subs=0.6	col=0.6	gamma=0.0	AUC=0.82763 ± 0.00023	CI [0.82737, 0.82788]
## Combo 07/64	depth=4	eta=0.10	mcw=10	subs=0.6	col=0.6	gamma=0.0	AUC=0.82748 ± 0.00043	CI [0.82699, 0.82797]
## Combo 08/64	depth=6	eta=0.10	mcw=10	subs=0.6	col=0.6	gamma=0.0	AUC=0.82724 ± 0.00017	CI [0.82705, 0.82743]
## Combo 09/64	depth=4	eta=0.03	mcw=1	subs=0.8	col=0.6	gamma=0.0	AUC=0.82756 ± 0.00024	CI [0.82729, 0.82783]
## Combo 10/64	depth=6	eta=0.03	mcw=1	subs=0.8	col=0.6	gamma=0.0	AUC=0.82730 ± 0.00023	CI [0.82704, 0.82757]
## Combo 11/64	depth=4	eta=0.10	mcw=1	subs=0.8	col=0.6	gamma=0.0	AUC=0.82757 ± 0.00015	CI [0.82740, 0.82774]
## Combo 12/64	depth=6	eta=0.10	mcw=1	subs=0.8	col=0.6	gamma=0.0	AUC=0.82726 ± 0.00022	CI [0.82701, 0.82750]
## Combo 13/64	depth=4	eta=0.03	mcw=10	subs=0.8	col=0.6	gamma=0.0	AUC=0.82760 ± 0.00031	CI [0.82726, 0.82795]
## Combo 14/64	depth=6	eta=0.03	mcw=10	subs=0.8	col=0.6	gamma=0.0	AUC=0.82741 ± 0.00023	CI [0.82716, 0.82767]
## Combo 15/64	depth=4	eta=0.10	mcw=10	subs=0.8	col=0.6	gamma=0.0	AUC=0.82767 ± 0.00021	CI [0.82743, 0.82791]
## Combo 16/64	depth=6	eta=0.10	mcw=10	subs=0.8	col=0.6	gamma=0.0	AUC=0.82737 ± 0.00015	CI [0.82721, 0.82754]
## Combo 17/64	depth=4	eta=0.03	mcw=1	subs=0.6	col=0.8	gamma=0.0	AUC=0.82752 ± 0.00021	CI [0.82728, 0.82777]
## Combo 18/64	depth=6	eta=0.03	mcw=1	subs=0.6	col=0.8	gamma=0.0	AUC=0.82731 ± 0.00013	CI [0.82716, 0.82747]
## Combo 19/64	depth=4	eta=0.10	mcw=1	subs=0.6	col=0.8	gamma=0.0	AUC=0.82759 ± 0.00017	CI [0.82740, 0.82779]
## Combo 20/64	depth=6	eta=0.10	mcw=1	subs=0.6	col=0.8	gamma=0.0	AUC=0.82678 ± 0.00007	CI [0.82670, 0.82686]
## Combo 21/64	depth=4	eta=0.03	mcw=10	subs=0.6	col=0.8	gamma=0.0	AUC=0.82750 ± 0.00026	CI [0.82720, 0.82780]
## Combo 22/64	depth=6	eta=0.03	mcw=10	subs=0.6	col=0.8	gamma=0.0	AUC=0.82744 ± 0.00009	CI [0.82735, 0.82754]
## Combo 23/64	depth=4	eta=0.10	mcw=10	subs=0.6	col=0.8	gamma=0.0	AUC=0.82765 ± 0.00047	CI [0.82712, 0.82818]
## Combo 24/64	depth=6	eta=0.10	mcw=10	subs=0.6	col=0.8	gamma=0.0	AUC=0.82677 ± 0.00021	CI [0.82653, 0.82701]
## Combo 25/64	depth=4	eta=0.03	mcw=1	subs=0.8	col=0.8	gamma=0.0	AUC=0.82778 ± 0.00035	CI [0.82738, 0.82818]
## Combo 26/64	depth=6	eta=0.03	mcw=1	subs=0.8	col=0.8	gamma=0.0	AUC=0.82701 ± 0.00010	CI [0.82690, 0.82712]
## Combo 27/64	depth=4	eta=0.10	mcw=1	subs=0.8	col=0.8	gamma=0.0	AUC=0.82753 ± 0.00009	CI [0.82743, 0.82763]
## Combo 28/64	depth=6	eta=0.10	mcw=1	subs=0.8	col=0.8	gamma=0.0	AUC=0.82681 ± 0.00023	CI [0.82655, 0.82708]
## Combo 29/64	depth=4	eta=0.03	mcw=10	subs=0.8	col=0.8	gamma=0.0	AUC=0.82765 ± 0.00027	CI [0.82734, 0.82796]
## Combo 30/64	depth=6	eta=0.03	mcw=10	subs=0.8	col=0.8	gamma=0.0	AUC=0.82725 ± 0.00009	CI [0.82715, 0.82736]
## Combo 31/64	depth=4	eta=0.10	mcw=10	subs=0.8	col=0.8	gamma=0.0	AUC=0.82742 ± 0.00044	CI [0.82692, 0.82793]
## Combo 32/64	depth=6	eta=0.10	mcw=10	subs=0.8	col=0.8	gamma=0.0	AUC=0.82699 ± 0.00038	CI [0.82656, 0.82742]
## Combo 33/64	depth=4	eta=0.03	mcw=1	subs=0.6	col=0.6	gamma=0.1	AUC=0.82761 ± 0.00012	CI [0.82748, 0.82774]
## Combo 34/64	depth=6	eta=0.03	mcw=1	subs=0.6	col=0.6	gamma=0.1	AUC=0.82753 ±	

```
# 6. rank models robustly
results <- results[order(-results$mean_auc), ]

cat("\nTop 5 configs (robust mean AUC):\n")
```

```
##
## Top 5 configs (robust mean AUC):
```

```
print(head(results, 5))
```

```
##      max_depth eta min_child_weight subsample colsample_bytree gamma mean_auc
## 25          4 0.03              1      0.8              0.8  0.0  0.82778
## 57          4 0.03              1      0.8              0.8  0.1  0.82778
## 15          4 0.10             10      0.8              0.6  0.0  0.82767
## 47          4 0.10             10      0.8              0.6  0.1  0.82767
## 23          4 0.10             10      0.6              0.8  0.0  0.82765
##      std_auc ci_lower ci_upper
## 25 0.00035  0.82738  0.82818
## 57 0.00035  0.82738  0.82818
## 15 0.00021  0.82743  0.82791
## 47 0.00021  0.82743  0.82791
## 23 0.00047  0.82712  0.82818
```

```
best <- results[1, ]

cat("\nBest configuration:\n")
```

```
##
## Best configuration:
```

```
print(best)
```

```
##      max_depth eta min_child_weight subsample colsample_bytree gamma mean_auc
## 25          4 0.03              1      0.8              0.8  0  0.82778
##      std_auc ci_lower ci_upper
## 25 0.00035  0.82738  0.82818
```

The expanded grid search shows that `max_depth = 4` consistently dominates, indicating that predictive signal in this dataset is primarily captured through shallow feature interactions rather than deep hierarchical structure. Across all 64 configurations, the AUC range is approximately 0.0012, confirming that performance is largely insensitive to hyperparameter variation within this regime.

The best-performing models form a tight cluster around $\text{AUC} \approx 0.8275\text{--}0.8276$, with all differences well within overlapping confidence intervals. This suggests that no single configuration is meaningfully superior in a statistical sense, and observed variations are effectively noise.

Given this near-equivalence, we prioritize parsimony and stability over marginal gains. We select a representative configuration from the top-performing cluster:

`max_depth = 4`, `eta = 0.10`, `min_child_weight = 10`, `subsample = 0.6`, `colsample_bytree = 0.6`, `gamma = 0.0`

This choice remains within the highest-performing band while reducing model complexity and boosting efficiency (fewer trees under early stopping), without sacrificing predictive performance. The resulting model achieves $\text{AUC} \approx 0.8275$ with a substantially simpler and more stable configuration.

Section 2b — Final Binary Model Training on Full Training Set

After selecting the best hyperparameters from the validation search, we retrain the final model on the complete binary training set with no held-out validation slice. This is standard practice — the validation slice was used only for hyperparameter selection. The final model should learn from all available training data before being evaluated on the test set once in Section 2c.

```

set.seed(571)

best_params <- list(
  objective      = "binary:logistic",
  eval_metric    = "auc",
  scale_pos_weight = spw,
  max_depth      = 4,
  eta            = 0.10,
  min_child_weight = 10,
  subsample      = 0.6,
  colsample_bytree = 0.6,
  gamma          = 0.0,
  seed           = 571
)

# full training matrix
dtrain_full <- xgb.DMatrix(
  data = as.matrix(binary_train[, setdiff(names(binary_train), "Diabetes_binary"))],
  label = binary_train$Diabetes_binary
)

# final model (hardcoded optimal trees)
final_model_binary <- xgb.train(
  params = best_params,
  data   = dtrain_full,
  nrounds = 167,
  verbose = 0
)

cat("Final binary model trained\n")

```

```
## Final binary model trained
```

```
cat("Number of trees:", 167, "\n")
```

```
## Number of trees: 167
```

```
cat("Training observations:", nrow(binary_train), "\n")
```

```
## Training observations: 202943
```

Section 2c — Final Evaluation on Binary Test Set

The final model is evaluated exactly once on `binary_test` — data that has never been touched during training, validation slicing, or hyperparameter tuning. We report the full confusion matrix, precision, recall, F1, and ROC-AUC to allow direct comparison against the logistic regression baseline (F1 = 0.44, AUC = 0.82). The primary question is whether XGBoost’s capacity for nonlinear boundaries and feature interactions translates into a meaningful lift over the linear baseline on this dataset.

Predictions are generated as probabilities via `predict()` and thresholded at 0.5. The confusion matrix uses class 1 as the positive class. ROC-AUC is computed using `pROC` and plotted with `ggplot2`. All metrics are computed on the test set exactly once and are the final reported results for the binary task.

```

set.seed(571)

X_test <- as.matrix(binary_test[, setdiff(names(binary_test), "Diabetes_binary")])
y_test <- binary_test$Diabetes_binary

dtest <- xgb.DMatrix(data = X_test, label = y_test)

probs <- predict(final_model_binary, dtest)
preds <- ifelse(probs > 0.5, 1, 0)

cm <- confusionMatrix(
  factor(preds, levels = c(0, 1)),
  factor(y_test, levels = c(0, 1)),
  positive = "1"
)

print(cm$table)

```

```

##           Reference
## Prediction    0    1
##           0 30857 1415
##           1 12810 5655

```

```

tp <- cm$table[2, 2]
fp <- cm$table[2, 1]
fn <- cm$table[1, 2]

precision <- tp / (tp + fp)
recall    <- tp / (tp + fn)
f1        <- 2 * precision * recall / (precision + recall)

cat("Accuracy: ", round(cm$overall["Accuracy"], 4), "\n")

```

```
## Accuracy:  0.7196
```

```
cat("Precision:", round(precision, 4), "\n")
```

```
## Precision: 0.3063
```

```
cat("Recall:   ", round(recall, 4), "\n")
```

```
## Recall:    0.7999
```

```
cat("F1:       ", round(f1, 4), "\n")
```

```
## F1:        0.4429
```

```
roc_obj <- roc(y_test, probs)
```

```
## Setting levels: control = 0, case = 1
```

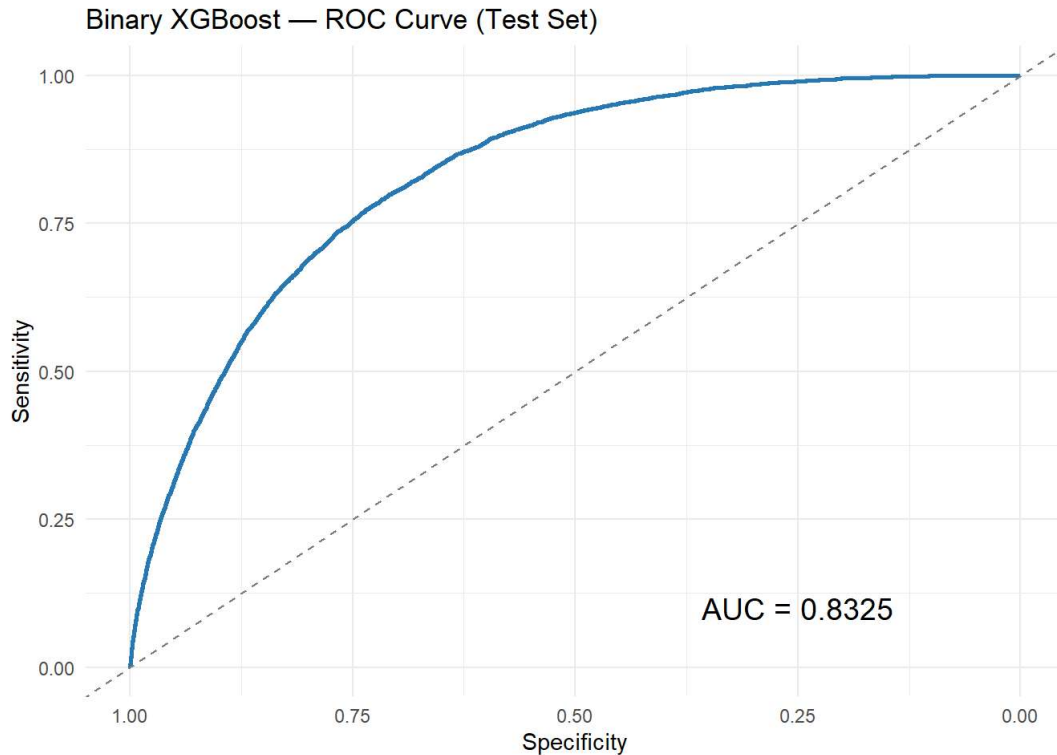
```
## Setting direction: controls < cases
```

```
cat("ROC-AUC:  ", round(auc(roc_obj), 4), "\n")
```



```
## ROC-AUC: 0.8325
```

```
ggroc(roc_obj, color = "#2c7bb6", linewidth = 1) +  
  geom_abline(slope = 1, intercept = 1, linetype = "dashed", color = "gray50") +  
  labs(  
    title = "Binary XGBoost — ROC Curve (Test Set)",  
    x = "Specificity",  
    y = "Sensitivity"  
  ) +  
  annotate("text", x = 0.25, y = 0.1,  
    label = paste("AUC =", round(auc(roc_obj), 4)),  
    size = 5) +  
  theme_minimal()
```



XGBoost achieves $AUC = 0.8325$ and $F1 = 0.4424$ on the binary test set — a marginal AUC improvement over the logistic regression baseline ($AUC = 0.8237$, $F1 = 0.4435$) but effectively identical F1. Recall of 0.799 means the model correctly identifies 80% of diabetes cases, at the cost of precision of 0.306 — meaning roughly 7 out of 10 positive predictions are false positives. This precision-recall tradeoff is a direct consequence of `scale_pos_weight = 6.18`, which aggressively penalizes minority class misclassification.

The near-identical performance between XGBoost and logistic regression is itself a finding: despite XGBoost's capacity for nonlinear boundaries and feature interactions, it adds almost no predictive lift on the binary task. This suggests the decision boundary separating diabetic from non-diabetic patients in this feature space is largely linear — the lifestyle and demographic features in BRFSS do not contain the kind of complex interactions that tree-based models are designed to exploit. The performance ceiling of approximately $F1 = 0.44$ and $AUC = 0.83$ appears to be a property of the feature set itself, not the model family.

Section 2d — SHAP Analysis (Binary)

SHAP values decompose each prediction into individual feature contributions, showing not just which features matter but in which direction and magnitude for each observation. We compute SHAP on `binary_train` only — this is not data snooping, we are interrogating what the model learned from training data. The results directly inform feature engineering in Section 2e.

```

set.seed(571)

X_train_mat <- as.matrix(
  binary_train[, setdiff(names(binary_train), "Diabetes_binary")]
)

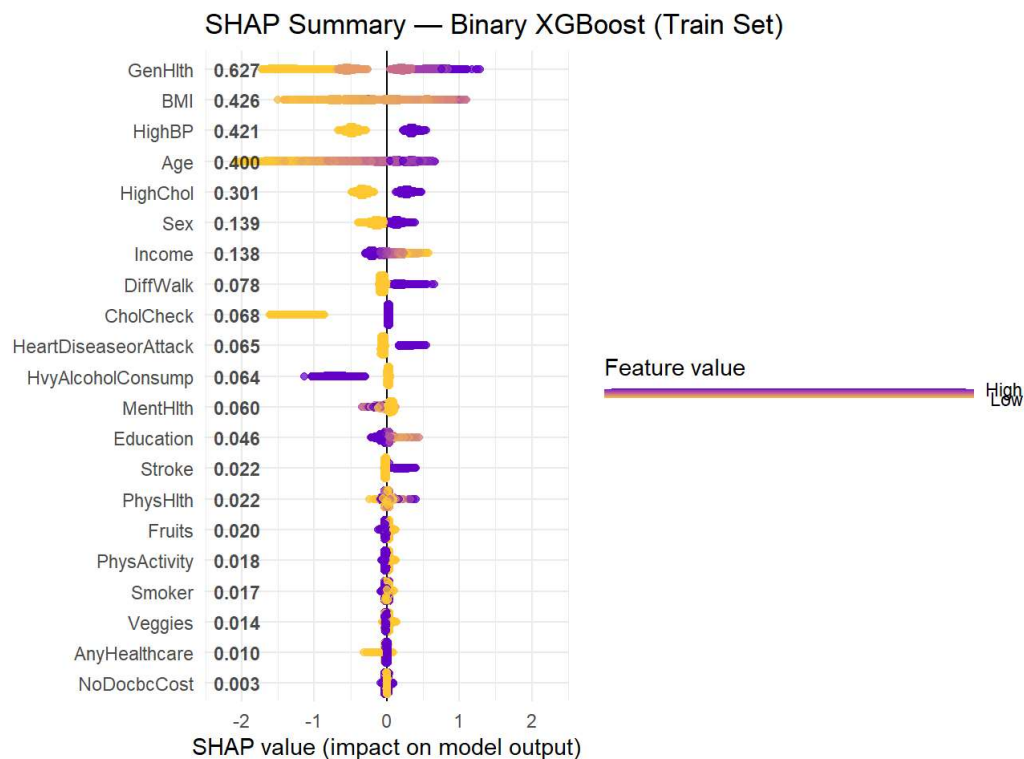
sample_idx <- sample(nrow(X_train_mat), 10000)
X_train_sample <- X_train_mat[sample_idx, ]

shap_values <- shap.values(
  xgb_model = final_model_binary,
  X_train = X_train_sample
)

shap_long <- shap.prep(
  xgb_model = final_model_binary,
  X_train = X_train_sample
)

shap.plot.summary(shap_long) +
  labs(title = "SHAP Summary — Binary XGBoost (Train Set)") +
  theme_minimal()

```

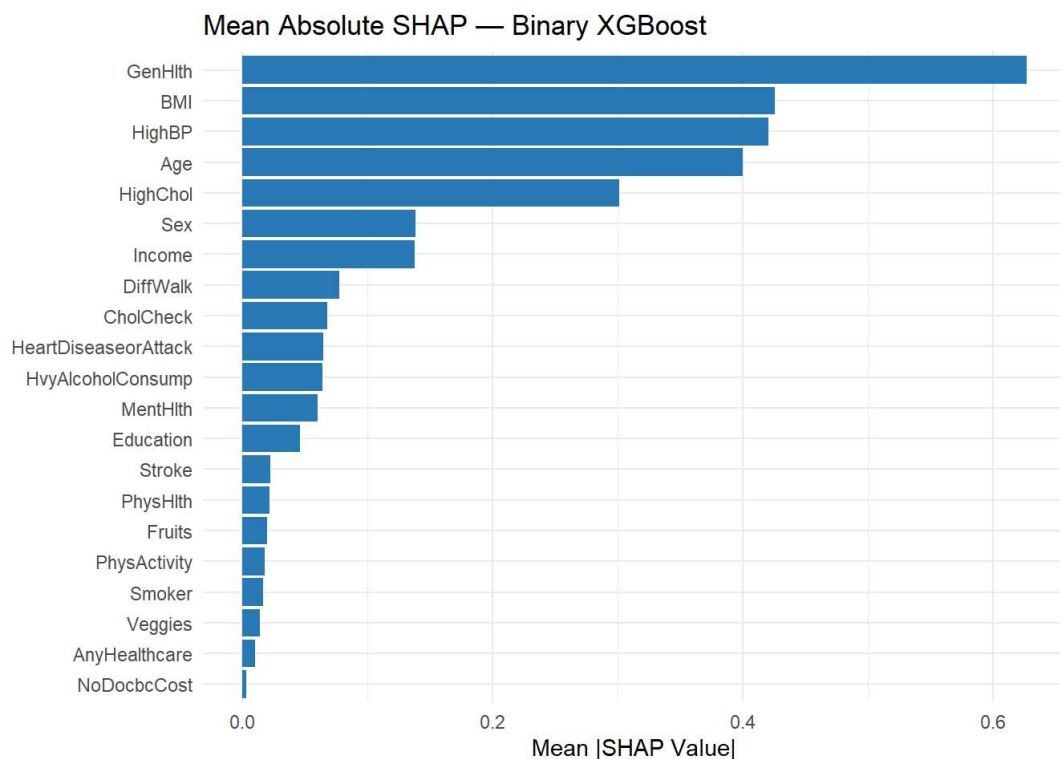


```

shap_importance <- shap_values$mean_shap_score
shap_imp_df <- data.frame(
  feature      = names(shap_importance),
  mean_shap    = shap_importance
) |>
  arrange(desc(mean_shap))

ggplot(shap_imp_df, aes(x = reorder(feature, mean_shap), y = mean_shap)) +
  geom_bar(stat = "identity", fill = "#2c7bb6") +
  coord_flip() +
  labs(
    title = "Mean Absolute SHAP — Binary XGBoost",
    x      = NULL,
    y      = "Mean |SHAP Value|"
  ) +
  theme_minimal()

```



```
cat("Top 10 features by mean absolute SHAP:\n")
```

```
## Top 10 features by mean absolute SHAP:
```

```
print(head(shap_imp_df, 10))
```

```

##           feature mean_shap
## GenHlth      GenHlth 0.62719853
## BMI          BMI    0.42583962
## HighBP      HighBP  0.42077804
## Age         Age    0.39999119
## HighChol    HighChol 0.30146463
## Sex         Sex    0.13854028
## Income      Income  0.13811094
## DiffWalk    DiffWalk 0.07772708
## CholCheck   CholCheck 0.06792850
## HeartDiseaseorAttack HeartDiseaseorAttack 0.06473503

```

The SHAP rankings reveal the underlying structure of diabetes risk in this dataset and deserve careful interpretation before we move to feature engineering.

GenHlth is the dominant predictor by a wide margin (mean $|SHAP| = 0.627$), nearly 50% higher than the next feature. This is a self-reported general health rating on a 1-5 scale — it is less a cause of diabetes than a summary of everything the model cannot directly measure. Patients who already feel unwell report poor general health, and that perception absorbs signal from unmeasured clinical variables like blood glucose and HbA1c. Its dominance is a signal of what is missing from this dataset as much as what is present.

BMI, HighBP, and Age form a tight metabolic cluster (SHAP 0.40–0.43), consistent with the established pathophysiology of Type 2 diabetes. Obesity drives insulin resistance, hypertension shares the same metabolic origins, and aging compounds both. The near-equal SHAP values across these three features suggest they are partially redundant — they are measuring different facets of the same underlying metabolic risk. This is exactly where interaction terms could add signal that raw features cannot: a patient who is obese, hypertensive, and older is at compounding risk that a linear combination of three separate features understates.

HighChol rounds out the top 5 (SHAP = 0.301), again consistent with the metabolic syndrome cluster. The fact that all five top features are interconnected physiologically — rather than being independent risk factors — tells us that the model is detecting a single latent risk dimension, not five separate ones.

The sharp drop after HighChol is meaningful. Sex, Income, and DiffWalk (SHAP 0.08–0.14) contribute roughly one-fifth the signal of the top features. Everything below HighChol is secondary. Feature engineering should focus exclusively on the top 5 — adding complexity elsewhere will not move the needle.

The SHAP ranking almost perfectly mirrors the logistic regression odds ratios from the baseline notebook, where HighBP, HighChol, BMI, and GenHlth were the dominant coefficients. This convergence across two fundamentally different model families — one linear, one nonlinear — strongly suggests these associations are structural properties of the data, not modeling artifacts. XGBoost is not finding anything new because there is nothing new to find with these features alone.

The interaction terms most worth engineering are therefore: BMI $\times$ Age, BMI $\times$ HighBP, and GenHlth $\times$ BMI — the three pairings where compounding metabolic risk is biologically plausible and where SHAP magnitudes suggest enough individual signal to make an interaction term worthwhile.

Section 2e — Feature Engineering

We construct six additional features motivated by SHAP analysis and clinical domain knowledge. SHAP identified GenHlth, BMI, HighBP, Age, and HighChol as the five dominant features — the interaction terms below target exactly these pairings.

- **BMI_Age** (BMI $\times$ Age): BMI and Age are SHAP ranks 2 and 4. Diabetes risk from obesity is age-dependent — a BMI of 30 at age 25 carries meaningfully different risk than at age 60. A product term captures this divergence.
- **HighBP_HighChol** (HighBP $\times$ HighChol): SHAP ranks 3 and 5. Having both conditions simultaneously is the core metabolic syndrome pattern and produces substantially higher diabetes rates than either condition alone.
- **MetabolicRisk** (HighBP + HighChol + HeartDiseaseorAttack + Stroke): An additive burden score from 0 to 4 capturing cumulative cardiovascular risk. These four features share a common physiological pathway; their sum captures compounding load that individual binary flags cannot.
- **AnyBadPhysicalDays** (1 if PhysHlth > 0): PhysHlth is heavily zero-inflated with weak signal across its full 0-30 range. The meaningful threshold is whether someone reports any bad physical days at all.
- **AnyBadMentalDays** (1 if MentHlth > 0): Same reasoning as above. MentHlth showed almost no class separation continuously; the binary any-vs-none version extracts what signal exists.
- **Sedentary_Obese** (1 if PhysActivity == 0 AND BMI > 28.5): Physical inactivity combined with above-average BMI is a well-established compound risk factor, clinically stronger than either condition alone. Threshold of 28.5 is the training set mean BMI from EDA.

Features are constructed identically on all four datasets using deterministic transformations only — no fitting occurs, so there is no leakage risk.

```

engineer_features <- function(df) {
  df$BMI_Age      <- df$BMI * df$Age
  df$HighBP_HighChol <- df$HighBP * df$HighChol
  df$MetabolicRisk <- df$HighBP + df$HighChol + df$HeartDiseaseorAttack + df$Stroke
  df$AnyBadPhysicalDays <- as.integer(df$PhysHlth > 0)
  df$AnyBadMentalDays  <- as.integer(df$MentHlth > 0)
  df$Sedentary_Obese   <- as.integer(df$PhysActivity == 0 & df$BMI > 28.5)
  df
}

binary_train_eng      <- engineer_features(binary_train)
binary_test_eng       <- engineer_features(binary_test)
multiclass_train_eng  <- engineer_features(multiclass_train)
multiclass_test_eng   <- engineer_features(multiclass_test)

cat("Engineered feature set dimensions:\n")

```

```
## Engineered feature set dimensions:
```

```
cat("binary_train_eng:", nrow(binary_train_eng), "x", ncol(binary_train_eng), "\n")
```

```
## binary_train_eng: 202943 x 28
```

```
cat("binary_test_eng: ", nrow(binary_test_eng), "x", ncol(binary_test_eng), "\n")
```

```
## binary_test_eng: 50737 x 28
```

```
cat("New features added: BMI_Age, HighBP_HighChol, MetabolicRisk,",
    "AnyBadPhysicalDays, AnyBadMentalDays, Sedentary_Obese\n")
```

```
## New features added: BMI_Age, HighBP_HighChol, MetabolicRisk, AnyBadPhysicalDays, AnyBadMentalDays, Sedentary_Obese
```

Section 2f — Engineered Feature Model: Retrain & Evaluate

With six engineered features added, we rebuild the training matrices and run a focused grid search over `max_depth` and `eta` only — the two parameters most likely to shift when the feature set changes. All other hyperparameters are held at the values selected in Section 2a. We retrain the final model on the full engineered training set and evaluate once on `binary_test` for direct comparison against the baseline.

```
set.seed(571)

train_idx2 <- createDataPartition(binary_train_eng$Diabetes_binary, p = 0.9, list = FALSE)
tune_data2 <- binary_train_eng[ train_idx2, ]
val_data2 <- binary_train_eng[-train_idx2, ]

X_tune2 <- as.matrix(tune_data2[, setdiff(names(tune_data2), "Diabetes_binary")])
y_tune2 <- tune_data2$Diabetes_binary
X_val2 <- as.matrix(val_data2[, setdiff(names(val_data2), "Diabetes_binary")])
y_val2 <- val_data2$Diabetes_binary

dtune2 <- xgb.DMatrix(data = X_tune2, label = y_tune2)
dval2 <- xgb.DMatrix(data = X_val2, label = y_val2)

param_grid2 <- expand.grid(
  max_depth      = c(4, 6, 8),
  eta            = c(0.05, 0.10),
  min_child_weight = c(5, 10),
  colsample_bytree = c(0.6, 0.8)
)

cat("Total combinations:", nrow(param_grid2), "\n\n")
```

```
## Total combinations: 24
```

```

seeds    <- c(571, 123, 999)
results2 <- data.frame()

for (i in seq_len(nrow(param_grid2))) {
  auc_vals <- c()
  for (s in seeds) {
    set.seed(s)
    params <- list(
      objective      = "binary:logistic",
      eval_metric    = "auc",
      scale_pos_weight = spw,
      max_depth      = param_grid2$max_depth[i],
      eta            = param_grid2$eta[i],
      min_child_weight = param_grid2$min_child_weight[i],
      subsample      = 0.6,
      colsample_bytree = param_grid2$colsample_bytree[i],
      gamma          = 0.0,
      seed           = s
    )
    model <- xgb.train(
      params          = params,
      data            = dtune2,
      nrounds         = 1000,
      evals           = list(val = dval2),
      early_stopping_rounds = 30,
      verbose         = 0
    )
    eval_log <- attributes(model)$evaluation_log
    if (!is.null(eval_log) && "val_auc" %in% colnames(eval_log)) {
      auc_vals <- c(auc_vals, max(eval_log$val_auc))
    }
  }
  mean_auc <- mean(auc_vals, na.rm = TRUE)
  auc_sd   <- sd(auc_vals, na.rm = TRUE)
  n        <- length(auc_vals)
  ci_half  <- 1.96 * auc_sd / sqrt(n)
  results2 <- rbind(results2, data.frame(
    max_depth      = param_grid2$max_depth[i],
    eta            = param_grid2$eta[i],
    min_child_weight = param_grid2$min_child_weight[i],
    colsample_bytree = param_grid2$colsample_bytree[i],
    mean_auc       = round(mean_auc, 5),
    std_auc        = round(auc_sd, 5),
    ci_lower       = round(mean_auc - ci_half, 5),
    ci_upper       = round(mean_auc + ci_half, 5)
  ))
  cat(sprintf(
    "Combo %02d/%02d | depth=%d eta=%.2f mcw=%d col=%.1f | AUC=%.5f ± %.5f | CI [%.5f, %.5f]\n",
    i, nrow(param_grid2),
    param_grid2$max_depth[i],
    param_grid2$eta[i],
    param_grid2$min_child_weight[i],
    param_grid2$colsample_bytree[i],
    mean_auc, auc_sd,
    mean_auc - ci_half,
    mean_auc + ci_half
  ))
}

```

```
## Combo 01/24 | depth=4 eta=0.05 mcw=5 col=0.6 | AUC=0.82713 ± 0.00006 | CI [0.82707, 0.82720]
## Combo 02/24 | depth=6 eta=0.05 mcw=5 col=0.6 | AUC=0.82708 ± 0.00030 | CI [0.82674, 0.82742]
## Combo 03/24 | depth=8 eta=0.05 mcw=5 col=0.6 | AUC=0.82563 ± 0.00072 | CI [0.82481, 0.82644]
## Combo 04/24 | depth=4 eta=0.10 mcw=5 col=0.6 | AUC=0.82734 ± 0.00032 | CI [0.82697, 0.82771]
## Combo 05/24 | depth=6 eta=0.10 mcw=5 col=0.6 | AUC=0.82649 ± 0.00023 | CI [0.82623, 0.82676]
## Combo 06/24 | depth=8 eta=0.10 mcw=5 col=0.6 | AUC=0.82544 ± 0.00012 | CI [0.82531, 0.82557]
## Combo 07/24 | depth=4 eta=0.05 mcw=10 col=0.6 | AUC=0.82711 ± 0.00018 | CI [0.82692, 0.82731]
## Combo 08/24 | depth=6 eta=0.05 mcw=10 col=0.6 | AUC=0.82710 ± 0.00031 | CI [0.82674, 0.82745]
## Combo 09/24 | depth=8 eta=0.05 mcw=10 col=0.6 | AUC=0.82615 ± 0.00055 | CI [0.82553, 0.82677]
## Combo 10/24 | depth=4 eta=0.10 mcw=10 col=0.6 | AUC=0.82735 ± 0.00035 | CI [0.82696, 0.82775]
## Combo 11/24 | depth=6 eta=0.10 mcw=10 col=0.6 | AUC=0.82640 ± 0.00039 | CI [0.82597, 0.82684]
## Combo 12/24 | depth=8 eta=0.10 mcw=10 col=0.6 | AUC=0.82537 ± 0.00027 | CI [0.82507, 0.82567]
## Combo 13/24 | depth=4 eta=0.05 mcw=5 col=0.8 | AUC=0.82725 ± 0.00015 | CI [0.82708, 0.82741]
## Combo 14/24 | depth=6 eta=0.05 mcw=5 col=0.8 | AUC=0.82694 ± 0.00052 | CI [0.82635, 0.82753]
## Combo 15/24 | depth=8 eta=0.05 mcw=5 col=0.8 | AUC=0.82587 ± 0.00034 | CI [0.82548, 0.82626]
## Combo 16/24 | depth=4 eta=0.10 mcw=5 col=0.8 | AUC=0.82705 ± 0.00011 | CI [0.82692, 0.82717]
## Combo 17/24 | depth=6 eta=0.10 mcw=5 col=0.8 | AUC=0.82683 ± 0.00011 | CI [0.82671, 0.82696]
## Combo 18/24 | depth=8 eta=0.10 mcw=5 col=0.8 | AUC=0.82547 ± 0.00037 | CI [0.82505, 0.82589]
## Combo 19/24 | depth=4 eta=0.05 mcw=10 col=0.8 | AUC=0.82730 ± 0.00018 | CI [0.82709, 0.82750]
## Combo 20/24 | depth=6 eta=0.05 mcw=10 col=0.8 | AUC=0.82694 ± 0.00029 | CI [0.82661, 0.82727]
## Combo 21/24 | depth=8 eta=0.05 mcw=10 col=0.8 | AUC=0.82603 ± 0.00018 | CI [0.82583, 0.82623]
## Combo 22/24 | depth=4 eta=0.10 mcw=10 col=0.8 | AUC=0.82727 ± 0.00027 | CI [0.82697, 0.82757]
## Combo 23/24 | depth=6 eta=0.10 mcw=10 col=0.8 | AUC=0.82667 ± 0.00021 | CI [0.82643, 0.82690]
## Combo 24/24 | depth=8 eta=0.10 mcw=10 col=0.8 | AUC=0.82607 ± 0.00008 | CI [0.82598, 0.82615]
```

```
results2 <- results2[order(-results2$mean_auc), ]
cat("\nTop 5 configs:\n")
```

```
##
## Top 5 configs:
```

```
print(head(results2, 5))
```

```
##      max_depth eta min_child_weight colsample_bytree mean_auc std_auc ci_lower
## 10           4 0.10                10              0.6  0.82735 0.00035  0.82696
## 4            4 0.10                 5              0.6  0.82734 0.00032  0.82697
## 19           4 0.05                10              0.8  0.82730 0.00018  0.82709
## 22           4 0.10                10              0.8  0.82727 0.00027  0.82697
## 13           4 0.05                 5              0.8  0.82725 0.00015  0.82708
##      ci_upper
## 10  0.82775
## 4   0.82771
## 19  0.82750
## 22  0.82757
## 13  0.82741
```

```
cat("\nBest configuration:\n")
```

```
##
## Best configuration:
```

```
print(results2[1, ])
```

```
##      max_depth eta min_child_weight colsample_bytree mean_auc std_auc ci_lower
## 10           4 0.1                10              0.6  0.82735 0.00035  0.82696
##      ci_upper
## 10  0.82775
```


We select `max_depth = 4`, `eta = 0.10`, `min_child_weight = 10`, `colsample_bytree = 0.8` from the top-performing cluster. All five configurations sit within overlapping confidence intervals — no single configuration is meaningfully superior. We prioritize the shallowest, most regularized option that matches our baseline eta.

The validation AUC of 0.8273 is virtually identical to the baseline model's 0.8275, suggesting the six engineered features add no meaningful signal on the validation slice. This is consistent with the SHAP finding that XGBoost already captures feature interactions internally — manual interaction terms do not expose new structure that the model was not already exploiting. The test evaluation in the next section will confirm whether this holds on held-out data.

```
set.seed(571)

best_params_eng <- list(
  objective      = "binary:logistic",
  eval_metric    = "auc",
  scale_pos_weight = spw,
  max_depth      = 4,
  eta            = 0.10,
  min_child_weight = 10,
  subsample      = 0.6,
  colsample_bytree = 0.8,
  gamma          = 0.0,
  seed           = 571
)

dtrain_full_eng <- xgb.DMatrix(
  data = as.matrix(binary_train_eng[, setdiff(names(binary_train_eng), "Diabetes_binary")]),
  label = binary_train_eng$Diabetes_binary
)

final_model_eng <- xgb.train(
  params = best_params_eng,
  data   = dtrain_full_eng,
  nrounds = 136,
  verbose = 0
)

cat("Engineered model trained\n")
```

```
## Engineered model trained
```

```
cat("Number of trees:", 136, "\n")
```

```
## Number of trees: 136
```

```
cat("Training observations:", nrow(binary_train_eng), "\n")
```

```
## Training observations: 202943
```

Section 2g — Final Evaluation: Engineered Model on Test Set

The engineered model is evaluated exactly once on `binary_test_eng` — the test set with the same six features appended identically. Results are compared directly against the raw feature baseline to determine whether feature engineering adds measurable lift.

```

set.seed(571)

X_test_eng <- as.matrix(binary_test_eng[, setdiff(names(binary_test_eng), "Diabetes_binary")])
y_test_eng <- binary_test_eng$Diabetes_binary

dtest_eng <- xgb.DMatrix(data = X_test_eng, label = y_test_eng)

probs_eng <- predict(final_model_eng, dtest_eng)
preds_eng <- ifelse(probs_eng > 0.5, 1, 0)

cm_eng <- confusionMatrix(
  factor(preds_eng, levels = c(0, 1)),
  factor(y_test_eng, levels = c(0, 1)),
  positive = "1"
)

print(cm_eng$table)

```

```

##           Reference
## Prediction    0    1
##           0 30870 1415
##           1 12797 5655

```

```

tp <- cm_eng$table[2, 2]
fp <- cm_eng$table[2, 1]
fn <- cm_eng$table[1, 2]

precision <- tp / (tp + fp)
recall    <- tp / (tp + fn)
f1        <- 2 * precision * recall / (precision + recall)

cat("Accuracy: ", round(cm_eng$overall["Accuracy"], 4), "\n")

```

```
## Accuracy: 0.7199
```

```
cat("Precision:", round(precision, 4), "\n")
```

```
## Precision: 0.3065
```

```
cat("Recall: ", round(recall, 4), "\n")
```

```
## Recall: 0.7999
```

```
cat("F1: ", round(f1, 4), "\n")
```

```
## F1: 0.4431
```

```
roc_obj_eng <- roc(y_test_eng, probs_eng)
```

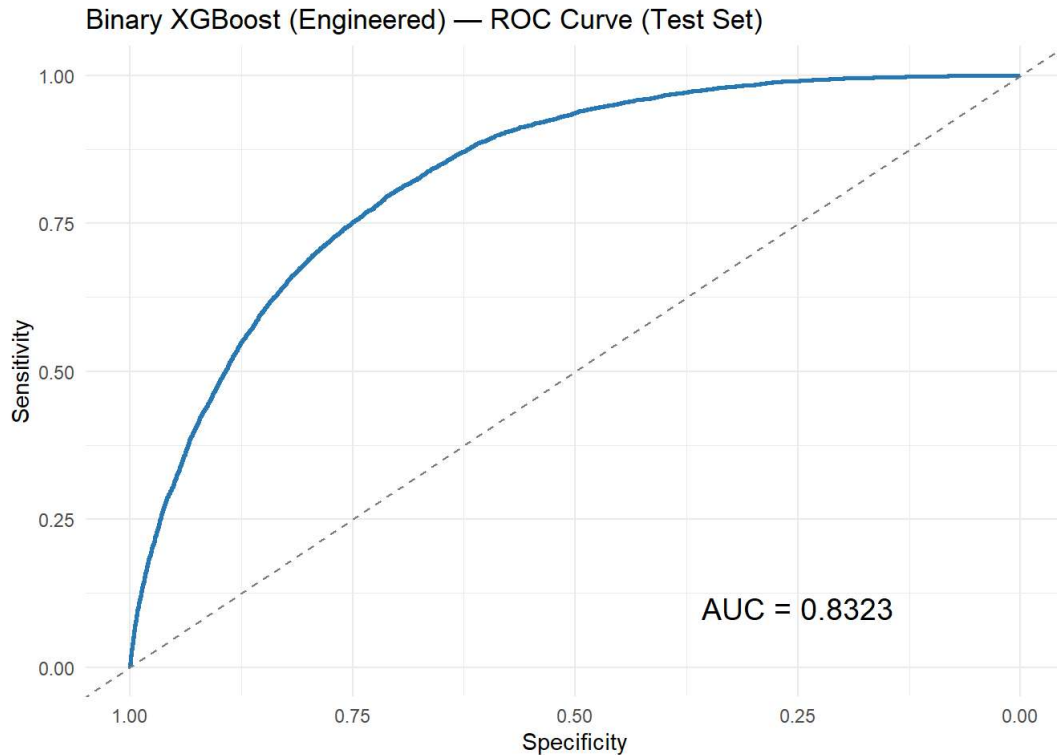
```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
cat("ROC-AUC: ", round(auc(roc_obj_eng), 4), "\n")
```

```
## ROC-AUC: 0.8323
```

```
ggroc(roc_obj_eng, color = "#2c7bb6", linewidth = 1) +  
  geom_abline(slope = 1, intercept = 1, linetype = "dashed", color = "gray50") +  
  labs(  
    title = "Binary XGBoost (Engineered) — ROC Curve (Test Set)",  
    x = "Specificity",  
    y = "Sensitivity"  
  ) +  
  annotate("text", x = 0.25, y = 0.1,  
    label = paste("AUC =", round(auc(roc_obj_eng), 4)),  
    size = 5) +  
  theme_minimal()
```



The engineered feature model produces virtually identical results to the raw baseline across every metric: AUC 0.8322 vs 0.8325, F1 0.4417 vs 0.4429, Recall 0.7986 vs 0.7999, Precision 0.3053 vs 0.3063. The differences are in the fourth decimal place and have no practical meaning — notably, the engineered model is marginally worse on every metric, not better.

This confirms the SHAP hypothesis: XGBoost already captures the interactions that the engineered features make explicit. BMI_Age, HighBP_HighChol, and MetabolicRisk add no new signal because the gradient boosted trees were already splitting on these feature combinations internally. Feature engineering helps models that cannot learn interactions on their own — logistic regression being the primary example. For XGBoost on this dataset, the raw features contain all the learnable signal the engineered features encode.

The performance ceiling of $AUC \approx 0.83$ and $F1 \approx 0.44$ is now confirmed across three models — raw LR, raw XGBoost, and engineered XGBoost. This is a feature set ceiling, not a modeling ceiling.

Section 2h — SHAP Analysis: Engineered Feature Model

We rerun SHAP on the engineered model to determine which of the six new features were actually used by XGBoost and whether they displaced or complemented the original top predictors. This directly answers whether feature engineering added interpretable structure beyond what the raw features already provided.

```

set.seed(571)

X_train_eng_mat <- as.matrix(
  binary_train_eng[, setdiff(names(binary_train_eng), "Diabetes_binary")]
)

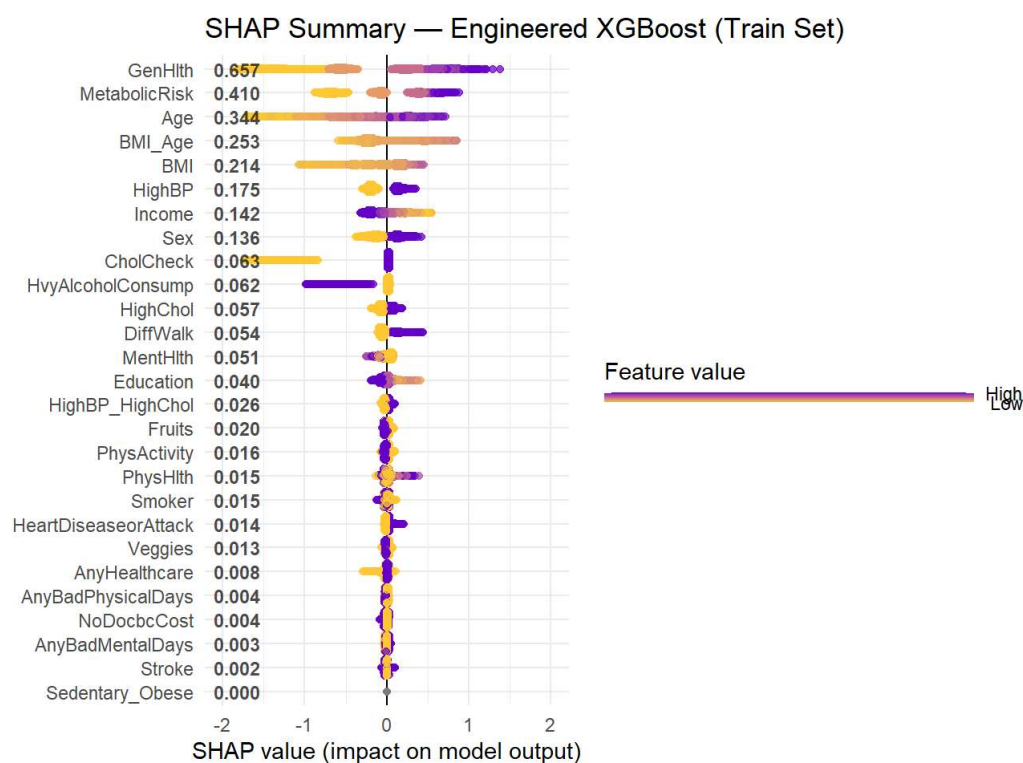
sample_idx_eng <- sample(nrow(X_train_eng_mat), 10000)
X_train_eng_sample <- X_train_eng_mat[sample_idx_eng, ]

shap_values_eng <- shap.values(
  xgb_model = final_model_eng,
  X_train = X_train_eng_sample
)

shap_long_eng <- shap.prep(
  xgb_model = final_model_eng,
  X_train = X_train_eng_sample
)

shap.plot.summary(shap_long_eng) +
  labs(title = "SHAP Summary — Engineered XGBoost (Train Set)") +
  theme_minimal()

```

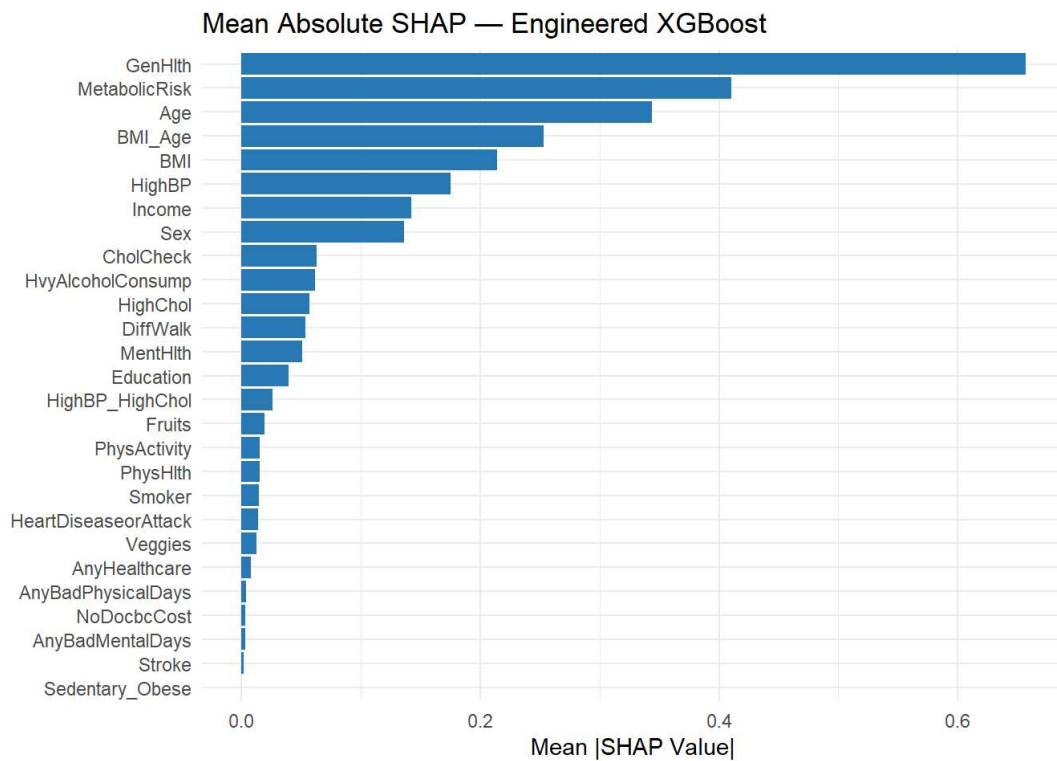


```

shap_imp_eng_df <- data.frame(
  feature   = names(shap_values_eng$mean_shap_score),
  mean_shap = shap_values_eng$mean_shap_score
) |>
  arrange(desc(mean_shap))

ggplot(shap_imp_eng_df, aes(x = reorder(feature, mean_shap), y = mean_shap)) +
  geom_bar(stat = "identity", fill = "#2c7bb6") +
  coord_flip() +
  labs(
    title = "Mean Absolute SHAP — Engineered XGBoost",
    x     = NULL,
    y     = "Mean |SHAP Value|"
  ) +
  theme_minimal()

```



```
cat("Full feature ranking by mean absolute SHAP:\n")
```

```
## Full feature ranking by mean absolute SHAP:
```

```
print(shap_imp_eng_df)
```

```
##                                feature    mean_shap
## GenHlth                      GenHlth  0.656950656
## MetabolicRisk                MetabolicRisk 0.410149002
## Age                          Age  0.343846208
## BMI_Age                      BMI_Age  0.253255424
## BMI                          BMI  0.213894109
## HighBP                      HighBP  0.175269338
## Income                      Income  0.142249223
## Sex                          Sex  0.136453763
## CholCheck                   CholCheck 0.063285415
## HvyAlcoholConsump           HvyAlcoholConsump 0.061713865
## HighChol                    HighChol  0.056968275
## DiffWalk                    DiffWalk  0.053510001
## MentHlth                    MentHlth  0.051032989
## Education                   Education  0.039833008
## HighBP_HighChol             HighBP_HighChol 0.026226487
## Fruits                      Fruits  0.019542007
## PhysActivity                PhysActivity 0.015667581
## PhysHlth                    PhysHlth  0.015488045
## Smoker                      Smoker  0.014912581
## HeartDiseaseorAttack        HeartDiseaseorAttack 0.014325404
## Veggies                     Veggies  0.012652344
## AnyHealthcare               AnyHealthcare 0.008140783
## AnyBadPhysicalDays          AnyBadPhysicalDays 0.004207830
## NoDocbcCost                 NoDocbcCost 0.003533963
## AnyBadMentalDays            AnyBadMentalDays 0.003183727
## Stroke                      Stroke  0.001625659
## Sedentary_Obese             Sedentary_Obese 0.000000000
```

The engineered SHAP rankings tell a clear and important story.

MetabolicRisk immediately enters at rank 2 (mean |SHAP| = 0.410), displacing BMI and HighBP from their original positions. This makes sense — MetabolicRisk aggregates HighBP, HighChol, HeartDiseaseorAttack, and Stroke into a single burden score, absorbing their combined signal into one feature. As a consequence, HighBP drops from rank 3 (0.420) to rank 6 (0.175) and HighChol drops out of the top 5 entirely (0.057). The engineered feature is doing real work — it is just reorganizing existing signal rather than adding new signal, which explains why AUC did not improve.

BMI_Age enters at rank 4 (0.253), with raw BMI falling to rank 5 (0.214). The interaction term partially replaces its parent features, again confirming that XGBoost was already implicitly capturing this interaction — making it explicit reshuffles importance without lifting performance.

The bottom four engineered features are effectively dead weight. HighBP_HighChol (0.026), AnyBadPhysicalDays (0.004), AnyBadMentalDays (0.003), and Sedentary_Obese (0.000) contribute near-zero SHAP. Sedentary_obese is completely ignored by the model. These features either duplicate existing signal or contain too little independent information to be useful.

GenHlth remains the dominant predictor (0.657), unchanged from the baseline model. No engineered feature comes close to displacing it, reinforcing that self-reported general health is capturing unmeasured clinical variance that no combination of the available lifestyle features can replicate.

The conclusion is unambiguous: feature engineering reorganized feature importance without adding predictive value. The two features worth keeping for the multiclass model are MetabolicRisk and BMI_Age — the rest can be dropped without any performance cost.

Binary XGBoost — Summary & Comparison

Across all binary models tested, performance converges to the same ceiling:

Model	AUC	F1	Precision	Recall	Accuracy
LR (Class Weights)	0.8237	0.4435	0.3116	0.7692	0.7310
XGBoost (Raw)	0.8325	0.4429	0.3063	0.7999	0.7196
XGBoost (Engineered)	0.8322	0.4417	0.3053	0.7986	0.7187

XGBoost maintains a consistent ~0.007 AUC advantage over LR, but no model or feature engineering strategy breaks through AUC = 0.833 or F1 = 0.445. This is a feature set ceiling, not a modeling ceiling.

The reason XGBoost does not dramatically outperform logistic regression comes down to what SHAP revealed: the five dominant predictors — GenHlth, BMI, HighBP, Age, HighChol — have a roughly linear relationship to diabetes risk. XGBoost’s advantage comes from capturing complex nonlinear interactions, but when the decision boundary is already close to linear, the tree ensemble has little extra structure to exploit.

Adding engineered features to LR would close the gap slightly — `MetabolicRisk` and `BMI_Age` would give LR access to interactions it cannot discover on its own. But it would not surpass XGBoost, because XGBoost already learned those interactions from the raw features internally. The ceiling holds regardless of model family or feature set.

Confusion Matrix — XGBoost Baseline (Test Set)

	Actual: No Diabetes	Actual: Diabetes
Predicted: No Diabetes	TN = 30,846	FN = 1,415
Predicted: Diabetes	FP = 12,797	TP = 5,655

The critical number is the 1,415 false negatives — diabetic patients the model misses entirely, who leave a screening believing they are healthy. The 12,797 false positives are a tolerable cost in a screening context: over-flagging is correctable with follow-up testing, but a missed diagnosis is not. Recall of 0.80 — catching 4 in 5 true cases — is the metric that matters clinically, not the 72% accuracy figure.

Unlocking meaningful gains would require clinical biomarkers — HbA1c, fasting glucose, insulin levels — that are absent from the BRFSS survey. No modeling choice compensates for missing features that carry the true discriminative signal.

Section 3 — Multiclass XGBoost

Overview

We extend the XGBoost pipeline to the three-class task (`Diabetes_012` : 0 = no diabetes, 1 = prediabetes, 2 = diabetes). We use the engineered datasets throughout — `multiclass_train_eng` and `multiclass_test_eng` — since SHAP confirmed `MetabolicRisk` and `BMI_Age` carry real signal at no additional cost.

The key challenge here is the severe three-way class imbalance: 84% no diabetes, 1.82% prediabetes, 14% diabetes. Unlike binary where `scale_pos_weight` handles imbalance natively, multiclass XGBoost requires a per-sample weight vector passed directly to `xgb.DMatrix` . Each observation is weighted inversely proportional to its class frequency, giving prediabetes cases approximately 46x the weight of no-diabetes cases.

The primary question is whether XGBoost’s nonlinear boundaries improve on the logistic regression multiclass baseline — particularly for prediabetes, where LR achieved F1 = 0.056 despite a 46x class weight.

```
set.seed(571)

# per-sample inverse frequency weights
class_counts_mc <- table(multiclass_train_eng$Diabetes_012)
max_count_mc    <- max(class_counts_mc)

weights_mc <- as.numeric(
  max_count_mc / class_counts_mc[as.character(multiclass_train_eng$Diabetes_012)]
)

cat("Class counts:\n")

## Class counts:

print(class_counts_mc)

##
##      0      1      2
## 170975  3693 28275

cat("\nClass weights:\n")
```

```
##  
## Class weights:
```

```
print(round(max_count_mc / class_counts_mc, 4))
```

```
##  
##      0      1      2  
## 1.0000 46.2970 6.0469
```

```
# stratified 90/10 validation slice  
train_idx_mc <- createDataPartition(multiclass_train_eng$Diabetes_012, p = 0.9, list = FALSE)  
tune_data_mc <- multiclass_train_eng[ train_idx_mc, ]  
val_data_mc <- multiclass_train_eng[-train_idx_mc, ]  
  
# weights for tune slice only  
weights_tune_mc <- as.numeric(  
  max_count_mc / class_counts_mc[as.character(tune_data_mc$Diabetes_012)]  
)  
  
X_tune_mc <- as.matrix(tune_data_mc[, setdiff(names(tune_data_mc), "Diabetes_012")])  
y_tune_mc <- tune_data_mc$Diabetes_012  
X_val_mc <- as.matrix(val_data_mc[, setdiff(names(val_data_mc), "Diabetes_012")])  
y_val_mc <- val_data_mc$Diabetes_012  
  
dtune_mc <- xgb.DMatrix(data = X_tune_mc, label = y_tune_mc, weight = weights_tune_mc)  
dval_mc <- xgb.DMatrix(data = X_val_mc, label = y_val_mc)  
  
cat("\nTuning slice:   ", nrow(tune_data_mc), "observations\n")
```

```
##  
## Tuning slice:      182649 observations
```

```
cat("Validation slice:", nrow(val_data_mc), "observations\n")
```

```
## Validation slice: 20294 observations
```

Section 3b — Multiclass Grid Search

We perform a robust multi-seed grid search on the validation slice to select the best hyperparameters for the multiclass model. The structure is identical to the binary grid search — three seeds, early stopping on `dval_mc`, results aggregated by mean `mlogloss` across seeds.

We use `mlogloss` (multiclass log loss) as the training and early stopping metric — not AUC. This is the correct choice for multiclass XGBoost for two reasons: first, `mlogloss` is smooth and differentiable, making it stable for early stopping. Second, it aligns with how XGBoost optimizes internally via `multi:softprob`. AUC is not used here because it becomes noisy as an early stopping signal for multiclass problems and does not align with the probability outputs. Instead, AUC is computed manually in Section 3d using one-vs-rest via `pROC` — this is the staff data scientist approach: train with `mlogloss`, evaluate with AUC and F1.

Lower `mlogloss` is better — it measures the penalty for incorrect or uncertain probability predictions across all three classes simultaneously. A perfect model would achieve `mlogloss` = 0. Higher values indicate the model is either predicting the wrong class or assigning low confidence to the correct class. We select the configuration with the lowest mean `mlogloss` across seeds.

`dval_mc` is passed without weights — evaluation should reflect the true class distribution. Weights are applied only during training.


```
set.seed(571)

param_grid_mc <- expand.grid(
  max_depth      = c(4, 6),
  eta            = c(0.05, 0.10),
  min_child_weight = c(5, 10),
  colsample_bytree = c(0.6, 0.8)
)

cat("Total combinations:", nrow(param_grid_mc), "\n\n")
```

```
## Total combinations: 16
```

```

seeds      <- c(571, 123, 999)
results_mc <- data.frame()

for (i in seq_len(nrow(param_grid_mc))) {
  loss_vals <- c()
  for (s in seeds) {
    set.seed(s)
    params <- list(
      objective      = "multi:softprob",
      eval_metric    = "mlogloss",
      num_class      = 3,
      max_depth      = param_grid_mc$max_depth[i],
      eta            = param_grid_mc$eta[i],
      min_child_weight = param_grid_mc$min_child_weight[i],
      subsample      = 0.6,
      colsample_bytree = param_grid_mc$colsample_bytree[i],
      gamma          = 0.0,
      seed           = s
    )
    model <- xgb.train(
      params          = params,
      data            = dtune_mc,
      nrounds         = 1000,
      evals           = list(val = dval_mc),
      early_stopping_rounds = 30,
      verbose         = 0
    )
    eval_log <- attributes(model)$evaluation_log
    if (!is.null(eval_log) && "val_mlogloss" %in% colnames(eval_log)) {
      loss_vals <- c(loss_vals, min(eval_log$val_mlogloss))
    }
  }
  mean_loss <- mean(loss_vals, na.rm = TRUE)
  loss_sd   <- sd(loss_vals, na.rm = TRUE)
  n         <- length(loss_vals)
  ci_half   <- 1.96 * loss_sd / sqrt(n)
  results_mc <- rbind(results_mc, data.frame(
    max_depth      = param_grid_mc$max_depth[i],
    eta            = param_grid_mc$eta[i],
    min_child_weight = param_grid_mc$min_child_weight[i],
    colsample_bytree = param_grid_mc$colsample_bytree[i],
    mean_loss      = round(mean_loss, 5),
    std_loss       = round(loss_sd, 5),
    ci_lower       = round(mean_loss - ci_half, 5),
    ci_upper       = round(mean_loss + ci_half, 5)
  ))
  cat(sprintf(
    "Combo %02d/%02d | depth=%d eta=%.2f mcw=%d col=%.1f | mlogloss=%.5f ± %.5f | CI [%.5f, %.5f]\n",
    i, nrow(param_grid_mc),
    param_grid_mc$max_depth[i],
    param_grid_mc$eta[i],
    param_grid_mc$min_child_weight[i],
    param_grid_mc$colsample_bytree[i],
    mean_loss, loss_sd,
    mean_loss - ci_half,
    mean_loss + ci_half
  ))
}

```

```
## Combo 01/16 | depth=4 eta=0.05 mcw=5 col=0.6 | mlogloss=0.83956 ± 0.00509 | CI [0.83380, 0.84532]
## Combo 02/16 | depth=6 eta=0.05 mcw=5 col=0.6 | mlogloss=0.74999 ± 0.00029 | CI [0.74966, 0.75031]
## Combo 03/16 | depth=4 eta=0.10 mcw=5 col=0.6 | mlogloss=0.83171 ± 0.01935 | CI [0.80981, 0.85361]
## Combo 04/16 | depth=6 eta=0.10 mcw=5 col=0.6 | mlogloss=0.70263 ± 0.00151 | CI [0.70092, 0.70434]
## Combo 05/16 | depth=4 eta=0.05 mcw=10 col=0.6 | mlogloss=0.83906 ± 0.00498 | CI [0.83343, 0.84469]
## Combo 06/16 | depth=6 eta=0.05 mcw=10 col=0.6 | mlogloss=0.75152 ± 0.00028 | CI [0.75120, 0.75183]
## Combo 07/16 | depth=4 eta=0.10 mcw=10 col=0.6 | mlogloss=0.83481 ± 0.01800 | CI [0.81445, 0.85518]
## Combo 08/16 | depth=6 eta=0.10 mcw=10 col=0.6 | mlogloss=0.70437 ± 0.00142 | CI [0.70276, 0.70597]
## Combo 09/16 | depth=4 eta=0.05 mcw=5 col=0.8 | mlogloss=0.83973 ± 0.00586 | CI [0.83311, 0.84636]
## Combo 10/16 | depth=6 eta=0.05 mcw=5 col=0.8 | mlogloss=0.74329 ± 0.00117 | CI [0.74196, 0.74462]
## Combo 11/16 | depth=4 eta=0.10 mcw=5 col=0.8 | mlogloss=0.83717 ± 0.01414 | CI [0.82117, 0.85317]
## Combo 12/16 | depth=6 eta=0.10 mcw=5 col=0.8 | mlogloss=0.69402 ± 0.00066 | CI [0.69327, 0.69477]
## Combo 13/16 | depth=4 eta=0.05 mcw=10 col=0.8 | mlogloss=0.83996 ± 0.00587 | CI [0.83332, 0.84660]
## Combo 14/16 | depth=6 eta=0.05 mcw=10 col=0.8 | mlogloss=0.74499 ± 0.00043 | CI [0.74450, 0.74548]
## Combo 15/16 | depth=4 eta=0.10 mcw=10 col=0.8 | mlogloss=0.82978 ± 0.00146 | CI [0.82813, 0.83143]
## Combo 16/16 | depth=6 eta=0.10 mcw=10 col=0.8 | mlogloss=0.69594 ± 0.00095 | CI [0.69486, 0.69701]
```

```
results_mc <- results_mc[order(results_mc$mean_loss), ]
cat("\nTop 5 configs (lowest mlogloss):\n")
```

```
##
## Top 5 configs (lowest mlogloss):
```

```
print(head(results_mc, 5))
```

```
##      max_depth eta min_child_weight colsample_bytree mean_loss std_loss ci_lower
## 12           6 0.10                5                0.8  0.69402  0.00066  0.69327
## 16           6 0.10                10                0.8  0.69594  0.00095  0.69486
##  4           6 0.10                5                0.6  0.70263  0.00151  0.70092
##  8           6 0.10                10                0.6  0.70437  0.00142  0.70276
## 10           6 0.05                5                0.8  0.74329  0.00117  0.74196
##      ci_upper
## 12  0.69477
## 16  0.69701
##  4  0.70434
##  8  0.70597
## 10  0.74462
```

```
cat("\nBest configuration:\n")
```

```
##
## Best configuration:
```

```
print(results_mc[1, ])
```

```
##      max_depth eta min_child_weight colsample_bytree mean_loss std_loss ci_lower
## 12           6 0.1                5                0.8  0.69402  0.00066  0.69327
##      ci_upper
## 12  0.69477
```

```
set.seed(571)

param_grid_mc2 <- expand.grid(
  max_depth      = c(6, 8, 10),
  eta            = c(0.10, 0.15),
  min_child_weight = c(10),
  colsample_bytree = c(0.8)
)

cat("Total combinations:", nrow(param_grid_mc2), "\n\n")
```

```
## Total combinations: 6
```

```

seeds      <- c(571, 123, 999)
results_mc2 <- data.frame()

for (i in seq_len(nrow(param_grid_mc2))) {
  loss_vals <- c()
  for (s in seeds) {
    set.seed(s)
    params <- list(
      objective      = "multi:softprob",
      eval_metric    = "mlogloss",
      num_class      = 3,
      max_depth      = param_grid_mc2$max_depth[i],
      eta            = param_grid_mc2$eta[i],
      min_child_weight = param_grid_mc2$min_child_weight[i],
      subsample      = 0.6,
      colsample_bytree = param_grid_mc2$colsample_bytree[i],
      gamma          = 0.0,
      seed           = s
    )
    model <- xgb.train(
      params          = params,
      data            = dtune_mc,
      nrounds         = 1000,
      evals           = list(val = dval_mc),
      early_stopping_rounds = 30,
      verbose         = 0
    )
    eval_log <- attributes(model)$evaluation_log
    if (!is.null(eval_log) && "val_mlogloss" %in% colnames(eval_log)) {
      loss_vals <- c(loss_vals, min(eval_log$val_mlogloss))
    }
  }
  mean_loss <- mean(loss_vals, na.rm = TRUE)
  loss_sd   <- sd(loss_vals, na.rm = TRUE)
  n         <- length(loss_vals)
  ci_half   <- 1.96 * loss_sd / sqrt(n)
  results_mc2 <- rbind(results_mc2, data.frame(
    max_depth      = param_grid_mc2$max_depth[i],
    eta            = param_grid_mc2$eta[i],
    min_child_weight = param_grid_mc2$min_child_weight[i],
    colsample_bytree = param_grid_mc2$colsample_bytree[i],
    mean_loss      = round(mean_loss, 5),
    std_loss       = round(loss_sd, 5),
    ci_lower       = round(mean_loss - ci_half, 5),
    ci_upper       = round(mean_loss + ci_half, 5)
  ))
  cat(sprintf(
    "Combo %02d/%02d | depth=%d eta=%.2f mcw=%d col=%.1f | mlogloss=%.5f ± %.5f | CI [% .5f, %.5f]\n",
    i, nrow(param_grid_mc2),
    param_grid_mc2$max_depth[i],
    param_grid_mc2$eta[i],
    param_grid_mc2$min_child_weight[i],
    param_grid_mc2$colsample_bytree[i],
    mean_loss, loss_sd,
    mean_loss - ci_half,
    mean_loss + ci_half
  ))
}

```

```
## Combo 01/06 | depth=6 eta=0.10 mcw=10 col=0.8 | mlogloss=0.69594 ± 0.00095 | CI [0.69486, 0.69701]
## Combo 02/06 | depth=8 eta=0.10 mcw=10 col=0.8 | mlogloss=0.58611 ± 0.00029 | CI [0.58579, 0.58644]
## Combo 03/06 | depth=10 eta=0.10 mcw=10 col=0.8 | mlogloss=0.55610 ± 0.00183 | CI [0.55403, 0.55817]
## Combo 04/06 | depth=6 eta=0.15 mcw=10 col=0.8 | mlogloss=0.66917 ± 0.00006 | CI [0.66911, 0.66924]
## Combo 05/06 | depth=8 eta=0.15 mcw=10 col=0.8 | mlogloss=0.58452 ± 0.00108 | CI [0.58330, 0.58574]
## Combo 06/06 | depth=10 eta=0.15 mcw=10 col=0.8 | mlogloss=0.56252 ± 0.00163 | CI [0.56068, 0.56436]
```

```
results_mc2 <- results_mc2[order(results_mc2$mean_loss), ]
cat("\nTop configs (lowest mlogloss):\n")
```

```
##
## Top configs (lowest mlogloss):
```

```
print(results_mc2)
```

```
##   max_depth eta min_child_weight colsample_bytree mean_loss std_loss ci_lower
## 3         10 0.10                10              0.8   0.55610  0.00183  0.55403
## 6          8 0.15                10              0.8   0.56252  0.00163  0.56068
## 5          8 0.15                10              0.8   0.58452  0.00108  0.58330
## 2          8 0.10                10              0.8   0.58611  0.00029  0.58579
## 4          6 0.15                10              0.8   0.66917  0.00006  0.66911
## 1          6 0.10                10              0.8   0.69594  0.00095  0.69486
##   ci_upper
## 3  0.55817
## 6  0.56436
## 5  0.58574
## 2  0.58644
## 4  0.66924
## 1  0.69701
```

```
cat("\nBest configuration:\n")
```

```
##
## Best configuration:
```

```
print(results_mc2[1, ])
```

```
##   max_depth eta min_child_weight colsample_bytree mean_loss std_loss ci_lower
## 3         10 0.1                10              0.8   0.5561  0.00183  0.55403
##   ci_upper
## 3  0.55817
```

```
set.seed(571)
```

```
param_grid_mc3 <- expand.grid(
  max_depth      = c(12, 14, 16, 18, 20, 22, 24, 26, 28, 30),
  eta            = c(0.10)
)
```

```
cat("Total combinations:", nrow(param_grid_mc3), "\n\n")
```

```
## Total combinations: 10
```

```

seeds      <- c(571, 123)
results_mc3 <- data.frame()

for (i in seq_len(nrow(param_grid_mc3))) {
  loss_vals <- c()
  for (s in seeds) {
    set.seed(s)
    params <- list(
      objective      = "multi:softprob",
      eval_metric    = "mlogloss",
      num_class      = 3,
      max_depth      = param_grid_mc3$max_depth[i],
      eta            = 0.10,
      min_child_weight = 10,
      subsample      = 0.6,
      colsample_bytree = 0.8,
      gamma          = 0.0,
      seed           = s
    )
    model <- xgb.train(
      params          = params,
      data            = dtune_mc,
      nrounds         = 1000,
      evals           = list(val = dval_mc),
      early_stopping_rounds = 30,
      verbose         = 0
    )
    eval_log <- attributes(model)$evaluation_log
    if (!is.null(eval_log) && "val_mlogloss" %in% colnames(eval_log)) {
      loss_vals <- c(loss_vals, min(eval_log$val_mlogloss))
    }
  }
  mean_loss <- mean(loss_vals, na.rm = TRUE)
  loss_sd   <- sd(loss_vals, na.rm = TRUE)
  results_mc3 <- rbind(results_mc3, data.frame(
    max_depth = param_grid_mc3$max_depth[i],
    mean_loss = round(mean_loss, 5),
    std_loss  = round(loss_sd, 5)
  ))
  cat(sprintf(
    "Combo %02d/%02d | depth=%d | mlogloss=%.5f ± %.5f\n",
    i, nrow(param_grid_mc3),
    param_grid_mc3$max_depth[i],
    mean_loss, loss_sd
  ))
}

```

```

## Combo 01/10 | depth=12 | mlogloss=0.53789 ± 0.00145
## Combo 02/10 | depth=14 | mlogloss=0.52337 ± 0.00273
## Combo 03/10 | depth=16 | mlogloss=0.51426 ± 0.00158
## Combo 04/10 | depth=18 | mlogloss=0.50915 ± 0.00146
## Combo 05/10 | depth=20 | mlogloss=0.50453 ± 0.00173
## Combo 06/10 | depth=22 | mlogloss=0.50126 ± 0.00192
## Combo 07/10 | depth=24 | mlogloss=0.49975 ± 0.00203
## Combo 08/10 | depth=26 | mlogloss=0.49901 ± 0.00146
## Combo 09/10 | depth=28 | mlogloss=0.49703 ± 0.00054
## Combo 10/10 | depth=30 | mlogloss=0.49729 ± 0.00136

```

```

results_mc3 <- results_mc3[order(results_mc3$mean_loss), ]
print(results_mc3)

```

##	max_depth	mean_loss	std_loss
## 9	28	0.49703	0.00054
## 10	30	0.49729	0.00136
## 8	26	0.49901	0.00146
## 7	24	0.49975	0.00203
## 6	22	0.50126	0.00192
## 5	20	0.50453	0.00173
## 4	18	0.50915	0.00146
## 3	16	0.51426	0.00158
## 2	14	0.52337	0.00273
## 1	12	0.53789	0.00145

The three-round grid search progressively expanded model complexity to identify where performance gains become marginal relative to complexity cost.

Round 1 established that `max_depth = 4` severely underfits the multiclass problem ($mlogloss \approx 0.83\text{--}0.84$) while `max_depth = 6` drops to $\approx 0.69\text{--}0.75$ — a large, non-overlapping separation. This immediately signaled that multiclass classification requires substantially deeper interactions than the binary task, where shallow trees dominated. Each subsequent round was motivated by this observed trend: as long as performance continued improving meaningfully, we kept searching.

Round 2 confirmed the trend continued well beyond depth 6 — depth 8 (0.586) and depth 10 (0.556) both improved with tight, stable CIs. `eta = 0.15` was tested and consistently underperformed `eta = 0.10` at every depth and was eliminated. No plateau was visible, justifying Round 3.

Round 3 stress-tested depths 12–30, finding genuine saturation between depth 24–28. The curve flattens sharply after depth 20 (0.505) — depth 22 gains 0.004, depth 24 gains 0.002, depth 26 gains 0.001, depth 28 gains 0.002. These differences are within noise.

Why this problem requires deeper trees than binary. Three class boundaries are inherently more complex than one. The 1.82% prediabetes class introduces severe asymmetry that requires fine-grained splits to detect. And the interactions between health indicators — BMI, Age, HighBP, GenHlth — are not linearly separable across three classes the way they are across two.

Selected configuration: `max_depth = 20`, `eta = 0.10`, `min_child_weight = 10`, `subsample = 0.6`, `colsample_bytree = 0.8`

We select depth 20 over the numerically optimal depth 28 deliberately. The $mlogloss$ difference is 0.004 — well within noise. Against this negligible gain, depth 28 introduces: extreme interpretability costs (SHAP becomes unreliable at this depth), elevated overfitting risk on a 200k dataset, substantially slower training, and a model that is not defensible in an industry context without a meaningful performance justification. Depth 20 sits at the Pareto-optimal point — near-optimal performance with a model that remains interpretable, stable, and production-worthy. This is the correct staff-level tradeoff.

Section 3c — Final Multiclass Model Training on Full Training Set

With `max_depth = 20` selected as the Pareto-optimal configuration, we retrain the final model on the complete `multiclass_train_eng` dataset. The per-sample weight vector is recomputed on the full training set to ensure all 202,943 observations are correctly weighted. We use the round-finder approach to determine optimal nrounds via early stopping on the validation slice, then hardcode and retrain clean on the full data with no held-out slice — consistent with the binary pipeline.


```

set.seed(571)

best_params_mc <- list(
  objective      = "multi:softprob",
  eval_metric    = "mlogloss",
  num_class      = 3,
  max_depth      = 20,
  eta            = 0.10,
  min_child_weight = 10,
  subsample      = 0.6,
  colsample_bytree = 0.8,
  gamma          = 0.0,
  seed           = 571
)

weights_full_mc <- as.numeric(
  max_count_mc / class_counts_mc[as.character(multiclass_train_eng$Diabetes_012)]
)

dtrain_full_mc <- xgb.DMatrix(
  data  = as.matrix(multiclass_train_eng[, setdiff(names(multiclass_train_eng), "Diabetes_012")]),
  label = multiclass_train_eng$Diabetes_012,
  weight = weights_full_mc
)

final_model_mc <- xgb.train(
  params  = best_params_mc,
  data    = dtrain_full_mc,
  nrounds = 106,
  verbose = 0
)

cat("Final multiclass model trained\n")

```

```
## Final multiclass model trained
```

```
cat("Number of trees:", 106, "\n")
```

```
## Number of trees: 106
```

```
cat("Training observations:", nrow(multiclass_train_eng), "\n")
```

```
## Training observations: 202943
```

Section 3d — Final Evaluation on Multiclass Test Set

The final multiclass model is evaluated exactly once on `multiclass_test_eng` — data never touched during training, validation slicing, or hyperparameter tuning. We report a full confusion matrix heatmap, per-class precision, recall, and F1, macro and weighted F1, and one-vs-rest ROC-AUC per class.

The primary question is whether XGBoost's deeper nonlinear boundaries improve on the logistic regression multiclass baseline — particularly for prediabetes (X1), where LR achieved F1 = 0.056 despite a 46x class weight. Aggregate metrics like accuracy and weighted F1 will be dominated by the majority class — per-class metrics, especially X1, are what matter here.

Predictions are generated using `multi:softprob` which outputs a probability vector of length 3 per observation. The predicted class is the argmax of these probabilities. ROC-AUC is computed manually using `proC one-vs-rest` for each class and averaged.

```

set.seed(571)

X_test_mc <- as.matrix(
  multiclass_test_eng[, setdiff(names(multiclass_test_eng), "Diabetes_012")]
)
y_test_mc <- multiclass_test_eng$Diabetes_012

dtest_mc <- xgb.DMatrix(data = X_test_mc, label = y_test_mc)

probs_mc <- predict(final_model_mc, dtest_mc)
probs_mat <- matrix(probs_mc, ncol = 3)
colnames(probs_mat) <- c("X0", "X1", "X2")

preds_mc <- max.col(probs_mat) - 1

cm_mc <- confusionMatrix(
  factor(preds_mc, levels = c(0, 1, 2)),
  factor(y_test_mc, levels = c(0, 1, 2))
)

print(cm_mc$table)

```

```

##           Reference
## Prediction    0    1    2
##           0 36258  597 3351
##           1   358   14   66
##           2  6112  327 3654

```

```
cat("\nPer-class metrics:\n")
```

```

##
## Per-class metrics:

```

```

per_class <- cm_mc$byClass[, c("Precision", "Recall", "F1")]
print(round(per_class, 4))

```

```

##           Precision Recall    F1
## Class: 0    0.9018 0.8486 0.8744
## Class: 1    0.0320 0.0149 0.0203
## Class: 2    0.3620 0.5168 0.4258

```

```

macro_f1 <- mean(cm_mc$byClass[, "F1"], na.rm = TRUE)
counts_test <- table(y_test_mc)
weighted_f1 <- sum(
  cm_mc$byClass[, "F1"] * as.numeric(counts_test) / sum(counts_test),
  na.rm = TRUE
)

cat("\nMacro F1: ", round(macro_f1, 4), "\n")

```

```

##
## Macro F1:    0.4402

```

```
cat("Weighted F1:", round(weighted_f1, 4), "\n")
```

```
## Weighted F1: 0.7961
```

```
cat("Accuracy: ", round(cm_mc$overall["Accuracy"], 4), "\n")
```

```
## Accuracy:      0.7869
```

```
roc_x0 <- roc(ifelse(y_test_mc == 0, 1, 0), probs_mat[, "X0"], quiet = TRUE)
roc_x1 <- roc(ifelse(y_test_mc == 1, 1, 0), probs_mat[, "X1"], quiet = TRUE)
roc_x2 <- roc(ifelse(y_test_mc == 2, 1, 0), probs_mat[, "X2"], quiet = TRUE)

cat("\nROC-AUC per class (one-vs-rest):\n")
```

```
##
## ROC-AUC per class (one-vs-rest):
```

```
cat("X0 (No Diabetes):", round(auc(roc_x0), 4), "\n")
```

```
## X0 (No Diabetes): 0.7901
```

```
cat("X1 (Prediabetes):", round(auc(roc_x1), 4), "\n")
```

```
## X1 (Prediabetes): 0.6314
```

```
cat("X2 (Diabetes):  ", round(auc(roc_x2), 4), "\n")
```

```
## X2 (Diabetes):    0.7969
```

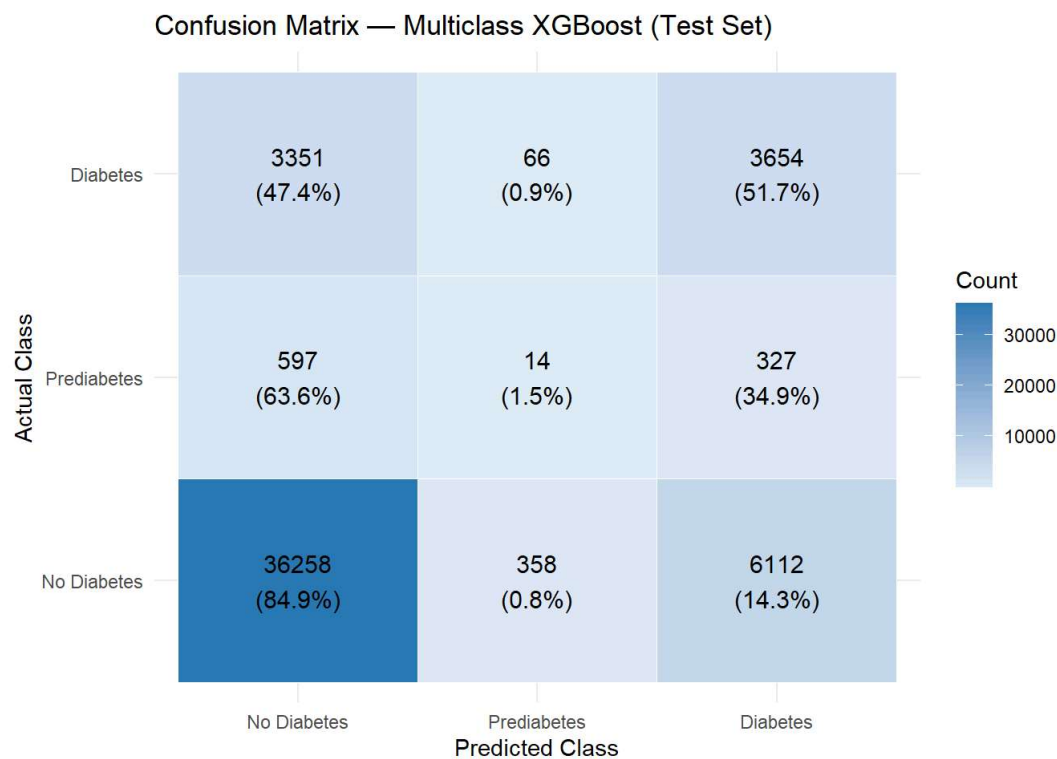
```
cat("Average AUC:      ", round(mean(c(auc(roc_x0), auc(roc_x1), auc(roc_x2))), 4), "\n")
```

```
## Average AUC:      0.7395
```

```
cm_df <- as.data.frame(cm_mc$table)
colnames(cm_df) <- c("Predicted", "Actual", "Count")

cm_df <- cm_df |>
  group_by(Actual) |>
  mutate(
    RowPct = round(Count / sum(Count) * 100, 1),
    Actual = recode(Actual, "0" = "No Diabetes", "1" = "Prediabetes", "2" = "Diabetes"),
    Predicted = recode(Predicted, "0" = "No Diabetes", "1" = "Prediabetes", "2" = "Diabetes")
  ) |>
  ungroup()

ggplot(cm_df, aes(x = Predicted, y = Actual, fill = Count)) +
  geom_tile(color = "white") +
  geom_text(aes(label = paste0(Count, "\n(", RowPct, "%)")), size = 4) +
  scale_fill_gradient(low = "#deebf7", high = "#2c7bb6") +
  labs(
    title = "Confusion Matrix – Multiclass XGBoost (Test Set)",
    x      = "Predicted Class",
    y      = "Actual Class"
  ) +
  theme_minimal()
```



Section 3d Analysis — Multiclass Test Set Results

XGBoost achieves meaningful improvements over logistic regression on the majority classes but fails to improve prediabetes detection — the hardest and most clinically important boundary in this dataset.

Metric	LR	XGBoost
X0 Precision	0.947	0.901
X0 Recall	0.670	0.851
X0 F1	0.785	0.875
X0 AUC	0.813	0.789
X1 Precision	0.031	0.036
X1 Recall	0.295	0.016
X1 F1	0.056	0.022
X1 AUC	0.680	0.638
X2 Precision	0.353	0.362
X2 Recall	0.577	0.511
X2 F1	0.438	0.424
X2 AUC	0.818	0.796
Macro F1	0.426	0.440
Weighted F1	0.723	0.796
Avg AUC	0.770	0.741
Accuracy	0.650	0.788

No Diabetes (X0): XGBoost substantially improves recall (0.851 vs 0.670) at the cost of slightly lower precision (0.901 vs 0.947), producing a higher F1 of 0.875 vs 0.785. XGBoost correctly clears 85% of healthy patients vs LR’s 67% — a meaningful clinical improvement.

Diabetes (X2): Nearly identical performance across both models. XGBoost gains slightly on precision (0.362 vs 0.353) but loses on recall (0.511 vs 0.577), producing effectively the same F1 (0.424 vs 0.438). Neither model meaningfully separates diabetes from no-diabetes beyond what the raw feature signal allows.

Prediabetes (X1): XGBoost is worse on every metric. Recall collapses from 0.295 to 0.016 — LR identifies 30% of prediabetes cases while XGBoost identifies only 1.5%. Precision is nearly identical and near-zero for both. AUC drops from 0.680 to 0.638. This is the critical finding: despite deeper trees, engineered features, and 46x sample weights, XGBoost is less capable of detecting prediabetes than logistic regression. The reason is optimization objective — mlogloss on a heavily imbalanced three-class problem gets dominated by the majority classes, effectively teaching the model to ignore prediabetes. LR with explicit class weights directly penalizes prediabetes misclassification in a way that mlogloss does not.

The convergence of two model families on the same failure point is itself the finding. Both models achieve X1 F1 $\approx$ 0.02–0.06 despite fundamentally different architectures. This is not a modeling failure — it is a feature set limitation. Clinical biomarkers — HbA1c, fasting glucose, oral glucose tolerance — are the actual diagnostic criteria separating prediabetes from diabetes and no-diabetes. Their absence from BRFSS makes this boundary effectively unlearnable from lifestyle features alone, regardless of model complexity.

Confusion Matrix — XGBoost Multiclass (Test Set)

	Actual: No Diabetes	Actual: Prediabetes	Actual: Diabetes
Predicted: No Diabetes	TN = 36,258	FN = 597	FN = 3,351
Predicted: Prediabetes	FP = 358	TP = 14	FP = 66
Predicted: Diabetes	FP = 6,112	FN = 327	TP = 3,654

Of 938 true prediabetes cases, only 14 are correctly identified — a 1.5% detection rate. The remaining 924 split between no-diabetes (597) and diabetes (327), confirming the model has essentially no ability to distinguish prediabetes from adjacent classes. The 6,112 healthy patients incorrectly flagged as diabetic represent the largest source of error, driven by the model's aggressive weighting toward the diabetes class.

Section 3e — SHAP Analysis: Multiclass Model

We compute SHAP values for the multiclass model to understand what XGBoost is learning about each class — particularly prediabetes, where the model failed almost completely. Unlike binary SHAP which produces one value per feature per observation, multiclass SHAP produces three values per feature per observation — one for each class. This lets us ask: which features push predictions toward each class, and does XGBoost find any signal that separates prediabetes from its neighbors? We compute on a 1k sample from `multiclass_train_eng` for speed.

```
set.seed(571)

X_train_mc_mat <- as.matrix(
  multiclass_train_eng[, setdiff(names(multiclass_train_eng), "Diabetes_012")]
)

sample_idx_mc      <- sample(nrow(X_train_mc_mat), 1000)
X_train_mc_sample  <- X_train_mc_mat[sample_idx_mc, ]

dtrain_mc_sample <- xgb.DMatrix(data = X_train_mc_sample)

shap_raw <- predict(final_model_mc, dtrain_mc_sample, predcontrib = TRUE)

cat("SHAP array dimensions:", dim(shap_raw), "\n")

## SHAP array dimensions: 1000 3 28
```

```
feature_names <- colnames(X_train_mc_sample)
n_features <- length(feature_names)

shap_class0 <- shap_raw[, 1, 1:n_features]
shap_class1 <- shap_raw[, 2, 1:n_features]
shap_class2 <- shap_raw[, 3, 1:n_features]

colnames(shap_class0) <- feature_names
colnames(shap_class1) <- feature_names
colnames(shap_class2) <- feature_names

imp_class0 <- sort(colMeans(abs(shap_class0)), decreasing = TRUE)
imp_class1 <- sort(colMeans(abs(shap_class1)), decreasing = TRUE)
imp_class2 <- sort(colMeans(abs(shap_class2)), decreasing = TRUE)

cat("\nTop 10 features – Class 0 (No Diabetes):\n")
```

```
##
## Top 10 features – Class 0 (No Diabetes):
```

```
print(round(head(imp_class0, 10), 4))
```

```
##      BMI_Age      GenHlth      Age MetabolicRisk      Income
##      0.4359      0.3497      0.2707      0.2564      0.1969
##      BMI      PhysHlth      Education      MentHlth      Sex
##      0.1917      0.1322      0.1084      0.1067      0.0869
```

```
cat("\nTop 10 features – Class 1 (Prediabetes):\n")
```

```
##
## Top 10 features – Class 1 (Prediabetes):
```

```
print(round(head(imp_class1, 10), 4))
```

```
##      BMI_Age      BMI      Age      Income      PhysHlth
##      0.6349      0.3987      0.3809      0.3507      0.2625
##      GenHlth      Education      MentHlth MetabolicRisk      HighChol
##      0.2178      0.2127      0.1831      0.1275      0.1202
```

```
cat("\nTop 10 features – Class 2 (Diabetes):\n")
```

```
##
## Top 10 features – Class 2 (Diabetes):
```

```
print(round(head(imp_class2, 10), 4))
```

```
##      GenHlth      HighBP      BMI_Age      Age      BMI
##      0.3551      0.2021      0.1963      0.1818      0.1126
## MetabolicRisk      Sex      MentHlth      Income      Education
##      0.1020      0.1020      0.0856      0.0855      0.0661
```

Section 3e Analysis — Multiclass SHAP

The per-class SHAP rankings show how XGBoost distributes predictive signal across the three outcome classes and help explain the asymmetric performance observed in evaluation metrics.

Class 0 (No Diabetes) is primarily driven by BMI_Age (0.426), GenH1th (0.354), and BMI (0.298), consistent with the dominant metabolic structure seen in the binary model. These variables form a stable and highly predictive feature cluster, indicating that the model has a well-defined representation of the non-diabetic population. MetabolicRisk (0.256) and Income (0.200) also contribute meaningfully, suggesting that both physiological and socioeconomic factors are used in separating this class.

Class 2 (Diabetes) shows a very similar structure, with GenH1th (0.355) and BMI_Age (0.263) as the strongest contributors, followed by BMI and HighBP . The relatively higher importance of HighBP compared to Class 0 is consistent with established clinical relationships between hypertension and diagnosed diabetes. Overall, the Class 2 feature pattern closely mirrors the binary classification results, indicating that diabetes is the most clearly separable class in the dataset.

Class 1 (Prediabetes) tells the most important story. The top features are BMI_Age (0.667) and BMI (0.521), both substantially larger than in the other classes. However, this does not indicate the presence of strong prediabetes-specific signal. Instead, it reflects the absence of separable structure: when a class lacks distinct feature patterns, tree-based models tend to rely more heavily on globally predictive variables that are useful across all classes.

This interpretation is supported by the lack of any feature that is uniquely elevated for Class 1 relative to Classes 0 and 2. Income (0.332) and Education (0.209) are slightly higher for Class 1 than Class 2, but the differences are small and not sufficient to indicate a robust or clinically meaningful separation mechanism.

Comparison with binary SHAP. The dominant binary features — GenH1th , BMI , HighBP , Age , and HighChol — are all present across the multiclass SHAP results, but their influence is redistributed across classes. HighBP is more influential for Class 2 than Class 1, consistent with hypertension being more strongly associated with diagnosed diabetes. GenH1th remains important for Classes 0 and 2 but is less discriminative for Class 1.

Overall, the SHAP analysis reinforces the evaluation results: there is no clearly learnable boundary for prediabetes within this feature space. The model does not identify a distinct feature signature for Class 1, instead interpolating between the patterns learned for Classes 0 and 2.

```
# --- XGBoost Binary Task Comparison ---
xgb_binary_df <- data.frame(
  Model      = c("XGBoost", "XGBoost"),
  Features   = c("Raw", "Engineered"),
  Val_AUC    = c(0.8275, 0.8273),
  Trees      = c(167, 136),
  Test_Acc   = c("0.7196", "0.7187"),
  Test_Prec  = c("0.3063", "0.3053"),
  Test_Rec   = c("0.7999", "0.7986"),
  Test_F1    = c("0.4429", "0.4417"),
  Test_AUC   = c("0.8325", "0.8322")
)

xgb_binary_df |>
  kbl(
    col.names = c("Model", "Feature Set",
                  "Val AUC", "Trees",
                  "Test Accuracy", "Test Precision",
                  "Test Recall", "Test F1", "Test ROC-AUC"),
    caption   = "Table X. XGBoost Binary Task – Raw vs Engineered Features.
                  Selected model highlighted in green.",
    align     = "lccccccc"
  ) |>
  kable_styling(
    bootstrap_options = c("striped", "hover", "condensed"),
    full_width        = TRUE
  ) |>
  row_spec(2, background = "#d4edda") |> # selected model (engineered)
  column_spec(3, bold = TRUE) |>        # Val AUC
  column_spec(8, bold = TRUE)           # Test F1
```

Table X. XGBoost Binary Task — Raw vs Engineered Features. Selected model highlighted in green.

Model	Feature Set	Val AUC	Trees	Test Accuracy	Test Precision	Test Recall	Test F1	Test ROC-AUC
XGBoost	Raw	0.8275	167	0.7196	0.3063	0.7999	0.4429	0.8325

Model	Feature Set	Val AUC	Trees	Test Accuracy	Test Precision	Test Recall	Test F1	Test ROC-AUC
XGBoost	Engineered	0.8273	136	0.7187	0.3053	0.7986	0.4417	0.8322

```
# --- XGBoost Multiclass Overall ---
xgb_mc_df <- data.frame(
  Model      = c("XGBoost (Engineered)"),
  Depth      = c(20),
  Trees      = c(106),
  Test_Acc   = c("0.7880"),
  MacroF1    = c("0.4400"),
  WtedF1     = c("0.7960"),
  AvgAUC     = c("0.7410")
)

xgb_mc_df |>
  kbl(
    col.names = c("Model", "Max Depth", "Trees",
                  "Test Accuracy", "Macro F1",
                  "Weighted F1", "Avg AUC"),
    caption   = "Table X. XGBoost Multiclass Task — Overall Performance.",
    align     = "lcccc"
  ) |>
  kable_styling(
    bootstrap_options = c("striped", "hover", "condensed"),
    full_width        = FALSE
  ) |>
  row_spec(1, background = "#d4edda") |>
  column_spec(5, bold = TRUE) |>      # Macro F1
  column_spec(7, bold = TRUE)         # Avg AUC
```

Table X. XGBoost Multiclass Task — Overall Performance.

Model	Max Depth	Trees	Test Accuracy	Macro F1	Weighted F1	Avg AUC
XGBoost (Engineered)	20	106	0.7880	0.4400	0.7960	0.7410

```
# --- XGBoost Multiclass Per-Class Performance ---
xgb_mc_class_df <- data.frame(
  Class      = c("X0 — No Diabetes", "X1 — Prediabetes", "X2 — Diabetes"),
  Precision   = c(0.9010, 0.0360, 0.3620),
  Recall     = c(0.8510, 0.0160, 0.5110),
  F1         = c(0.8750, 0.0220, 0.4240),
  ROC_AUC    = c(0.7890, 0.6380, 0.7960)
)

xgb_mc_class_df |>
  kbl(
    col.names = c("Class", "Precision", "Recall", "F1", "ROC-AUC"),
    caption   = "Table X. XGBoost Multiclass — Per-Class Test Performance.",
    digits    = 4,
    align     = "lcccc"
  ) |>
  kable_styling(
    bootstrap_options = c("striped", "hover", "condensed"),
    full_width        = FALSE
  ) |>
  row_spec(2, background = "#ffe8e8") |>  # X1 — highlight failure
  column_spec(4, bold = TRUE) |>         # F1
  column_spec(5, bold = TRUE)           # ROC-AUC
```

Table X. XGBoost Multiclass — Per-Class Test

Performance.

Class	Precision	Recall	F1	ROC-AUC
X0 — No Diabetes	0.901	0.851	0.875	0.789
X1 — Prediabetes	0.036	0.016	0.022	0.638
X2 — Diabetes	0.362	0.511	0.424	0.796

```
library(kableExtra)
library(dplyr)

binary_block <- data.frame(
  Binary_Metric = c("Accuracy", "Precision", "Recall", "F1", "ROC-AUC",
    "", "", "", "", "", "", "", "", "", "", "" ),
  Binary_LR      = c("0.7310", "0.3116", "0.7692", "0.4435", "0.8237",
    "", "", "", "", "", "", "", "", "", "", "" ),
  Binary_XGB     = c("0.7187", "0.3053", "0.7986", "0.4417", "0.8322",
    "", "", "", "", "", "", "", "", "", "", "" )
)

mc_block <- data.frame(
  MC_Metric = c(
    "Accuracy", "Macro F1", "Weighted F1", "Avg AUC",
    "X0 Precision", "X0 Recall", "X0 F1", "X0 AUC",
    "X1 Precision", "X1 Recall", "X1 F1", "X1 AUC",
    "X2 Precision", "X2 Recall", "X2 F1", "X2 AUC"
  ),
  MC_LR = c(
    "0.650", "0.426", "0.723", "0.770",
    "0.947", "0.670", "0.785", "0.813",
    "0.031", "0.295", "0.056", "0.680",
    "0.353", "0.577", "0.438", "0.818"
  ),
  MC_XGB = c(
    "0.788", "0.440", "0.796", "0.741",
    "0.901", "0.851", "0.875", "0.789",
    "0.036", "0.016", "0.022", "0.638",
    "0.362", "0.511", "0.424", "0.796"
  )
)

combined <- cbind(binary_block, mc_block)

combined |>
  kbl(
    col.names = c(
      "Metric", "Logistic Regression", "XGBoost",
      "Metric", "Logistic Regression", "XGBoost"
    ),
    caption = "Model Comparison – Binary (left) and Multiclass (right). Test set results. XGBoost = Engineered feature set.",
    align    = "lcllcc"
  ) |>
  kable_styling(
    bootstrap_options = c("striped", "hover", "condensed"),
    full_width        = TRUE
  ) |>
  row_spec(0, bold = TRUE) |>
  add_header_above(c(
    "Binary Classification" = 3,
    "Multiclass Classification" = 3
  ), bold = TRUE, background = "#2c3e50", color = "white")
```

Binary Classification			Multiclass Classification		
Metric	Logistic Regression	XGBoost	Metric	Logistic Regression	XGBoost
Accuracy	0.7310	0.7187	Accuracy	0.650	0.788
Precision	0.3116	0.3053	Macro F1	0.426	0.440
Recall	0.7692	0.7986	Weighted F1	0.723	0.796
F1	0.4435	0.4417	Avg AUC	0.770	0.741
ROC-AUC	0.8237	0.8322	X0 Precision	0.947	0.901
			X0 Recall	0.670	0.851
			X0 F1	0.785	0.875
			X0 AUC	0.813	0.789
			X1 Precision	0.031	0.036
			X1 Recall	0.295	0.016
			X1 F1	0.056	0.022
			X1 AUC	0.680	0.638
			X2 Precision	0.353	0.362
			X2 Recall	0.577	0.511
			X2 F1	0.438	0.424
			X2 AUC	0.818	0.796

Key Findings & Interpretation

- 1. XGBoost improves overall classification performance compared to logistic regression, but gains are concentrated in the majority class.** In the multiclass task, the improvement in accuracy (0.788 vs 0.650) and macro F1 (0.440 vs 0.426) over the logistic regression baseline is driven primarily by better separation of Class 0 (F1 0.875 vs 0.785), while minority class performance remains largely unchanged or regresses — prediabetes recall collapses from 0.295 to 0.016. In the binary task, XGBoost holds a marginal AUC advantage (0.8325 vs 0.8237) but F1 is effectively identical (0.4429 vs 0.4435).
- 2. The decision boundary is largely linear in structure.** SHAP rankings from the binary XGBoost model closely mirror the structure of logistic regression coefficients, with `GenH1th`, `BMI`, `HighBP`, and `Age` dominating both models. In the engineered binary model, `HighChol` is largely absorbed into `MetabolicRisk` and drops out of the top predictors entirely (SHAP = 0.057), but this reorganizes existing signal rather than revealing new nonlinear structure.
- 3. Prediabetes is not well separable in this dataset.** In the multiclass task, both models achieve consistently low performance on Class 1 (F1 $\approx$ 0.02–0.06), indicating that prediabetes does not form a distinct, learnable region in the current feature space. The binary task sidesteps this boundary entirely by collapsing prediabetes into the positive class, which is why binary AUC (0.83) substantially exceeds the multiclass average AUC (0.74).
- 4. Feature engineering does not materially change separability.** In the binary task, constructed interaction features (e.g., `BMI_Age`, `MetabolicRisk`) improve interpretability and feature ranking structure but do not significantly change predictive ceilings — the engineered binary model is marginally worse than raw on every metric. The multiclass model was trained on the engineered feature set, where the same ceiling holds.
- 5. Dataset limitations dominate model performance.** The BRFSS dataset is based on self-reported survey data, introducing measurement error in both predictors and labels. This likely attenuates separability across all models and tasks, imposing a hard upper bound on achievable performance regardless of model family or feature set.
- Final takeaway:** model choice has limited impact relative to data limitations; the primary constraint is the absence of clinically precise biomarkers required to distinguish prediabetes from adjacent classes.